VOLUMEN 04 · MÁSTER DATA & IA

Modelado analítico

El esquema en estrella: hechos, dimensiones, grano, claves y dimensiones que cambian con el tiempo. La capa gold de Marejada, lista para Power BI.

PARA QUIÉN

Álvaro, con el Volumen 03 terminado. Sabes responder casi cualquier pregunta con SQL; ahora vas a construir el modelo que las responde todas.

CÓMO ESTÁ ESCRITO

Primero las ideas, con ejemplos sobre tus datos; después la construcción de gold tabla a tabla, con cada consulta ejecutada de verdad; al final, cómo se usa el modelo y qué espera de él Power BI.

INCLUYE

El script de la capa gold y sus siete comprobaciones, ocho tickets con su solución y la revisión de Elena, y lo que el examen PL-300 pregunta sobre modelado.

Preparado para Álvaro · Octubre de 2026 · Un día y medio de trabajo

Índice

PARTE 0 · ANTES DE EMPEZAR 3

1. Cómo trabajar con este volumen 4

PARTE 1 · LAS IDEAS 6

2. Dos formas de organizar los mismos datos 7

3. Hechos y dimensiones 9

4. El grano 12

5. Las claves 14

PARTE 2 · CONSTRUIR GOLD 16

6. La dimensión fecha 17

7. Producto y comercial 20

8. Dimensiones que cambian: SCD 22

9. Las tablas de hechos 28

10. El script y sus comprobaciones 31

PARTE 3 · USAR EL MODELO 37

11. Consultar una estrella 38

12. Lo que Power BI espera del modelo 41

TICKETS · GOLD EN PRODUCCIÓN 43

Tickets · Volumen 04 44

Soluciones · Volumen 04 46

APÉNDICES · PARA TENER A MANO 55

A. Chuleta de modelado 56

B. Qué viene en el Volumen 05 58

PARTE 0

Antes de empezar

Por qué hace falta una tercera capa y qué vas a construir hoy.

1. Cómo trabajar con este volumen

1. Cómo trabajar con este volumen

1.1 De responder preguntas a construir el modelo

En los Volúmenes 02 y 03 has resuelto dieciocho tickets. Repásalos y verás que casi todos empiezan igual: unir `pedidos` con `lineas_pedido`, quitar los cancelados, unir con `clientes` por `cliente_maestro_id` para la provincia, con `productos` para la categoría, y agrupar por mes con `date_trunc`. Silver resolvió la limpieza, pero no la **forma**: cada pregunta obliga a recordar qué tablas unir, por qué columna y con qué filtros.

Eso tiene un coste que ahora va a crecer. A partir del Volumen 07, quien haga las preguntas no serás tú escribiendo SQL, sino Íñigo arrastrando campos en un informe de Power BI. Íñigo no sabe que hay que unir por `cliente_maestro_id` y no por `cliente_id`, ni que los cancelados no cuentan, ni que los portes no se pueden sumar desde las líneas. El modelo tiene que saberlo por él.

Esa es la función de la tercera capa, **gold**: datos organizados para que las preguntas de negocio se respondan solas, con una forma que todas las herramientas de análisis entienden. Esa forma tiene nombre: **esquema en estrella**.

1.2 Lo que vas a construir



TABLA	QUÉ ES	FILAS
`gold.fact_ventas`	Hechos: una línea de un pedido no cancelado	366.928
`gold.fact_pedidos`	Hechos: un pedido, en cualquier estado	77.448
`gold.dim_fecha`	Dimensión: un día de 2024 a 2026	1.096
`gold.dim_cliente`	Dimensión: un cliente real, con su historial de comercial asignado	1.349
`gold.dim_producto`	Dimensión: un producto del catálogo	236
`gold.dim_comercial`	Dimensión: una persona del equipo comercial	15

Las palabras "hechos", "dimensión", "grano" y "miembro desconocido" (la fila de más de cada dimensión) las vas a entender en la Parte 1. La Parte 2 construye las tablas una a una. La Parte 3 las usa.

1.3 Tu sesión de trabajo

Silver tiene que estar construida y comprobada, como siempre. Los tickets de este volumen van a su carpeta:

TÚ ESCRIBES

```
cd ~/master-data-ia
mkdir -p sql/04-modelado
duckdb db/marejada.duckdb < sql/02_comprobar_silver.sql
```

Y una novedad: este volumen deja **objetos permanentes** en tu base de datos (el esquema `gold`), así que añade dos scripts nuevos a la serie de bronze y silver: `sql/03_construir_gold.sql` y `sql/04_comprobar_gold.sql`. Los irás escribiendo a lo largo de la Parte 2; en el capítulo 10 los tendrás completos.

PARTE 1

Las ideas

Hechos, dimensiones, grano y claves: las cuatro decisiones que hay detrás de cualquier modelo analítico.

2. Dos formas de organizar los mismos datos

3. Hechos y dimensiones

4. El grano

5. Las claves

2. Dos formas de organizar los mismos datos

2.1 La forma del ERP

El ERP de Marejada guarda los datos **normalizados**: cada cosa en su tabla, sin repetir nada. El nombre de un cliente está una sola vez, en `clientes`; los pedidos solo llevan su `cliente_id`. Es lo que aprendiste en ASIR con las formas normales, y tiene todo el sentido para el trabajo de un ERP: miles de operaciones pequeñas en las que cambiar la dirección de un cliente debe ser modificar **una** fila, no cien mil.

Silver conserva esa forma: es el ERP limpio. Y por eso responder una pregunta de negocio sencilla en silver exige conocer la estructura. "Ventas de 2025 por provincia y categoría":

SQL · EN DUCKDB

```
SELECT c.provincia, pr.categoria, round(sum(l.importe_neto)) AS ventas
FROM silver.lineas_pedido AS l
JOIN silver.pedidos   AS p  ON p.pedido_id    = l.pedido_id
JOIN silver.clientes  AS c  ON c.cliente_id   = p.cliente_maestro_id
JOIN silver.productos AS pr ON pr.producto_id = l.producto_id
WHERE p.estado <> 'Cancelado'
  AND p.fecha >= '2025-01-01' AND p.fecha < '2026-01-01'
GROUP BY c.provincia, pr.categoria
ORDER BY ventas DESC
LIMIT 8;
```

RESULTADO

provincia varchar	categoria varchar	ventas double
Cantabria	Vinos	2395748.0
Cantabria	Cervezas	1948937.0
Cantabria	Alimentación	1425704.0
Cantabria	Refrescos	1282315.0
Asturias	Vinos	749135.0
Asturias	Cervezas	611474.0
Cantabria	Desechables	608634.0
Cantabria	Licores y destilados	583966.0

Tú lo escribes sin pensar, porque llevas dos volúmenes con estas tablas. Pero hay cuatro decisiones escondidas que cualquiera puede tomar mal: unir por `cliente_maestro_id` (si no, las fichas duplicadas cuentan aparte), quitar los cancelados, unir líneas con pedidos y no al revés, y usar `importe_neto` y no recalcularlo. Ahora imagina esas cuatro decisiones repartidas entre todas las personas que hacen informes en una empresa.

2.2 La forma analítica

El **modelado dimensional**, formalizado por Ralph Kimball en los años noventa y todavía la base de casi todos los modelos analíticos, organiza los datos alrededor de una idea: separar **lo que se mide** de **cómo se mira**.

- Lo que se mide son los **hechos**: las ventas, los pedidos, las unidades, los euros. Son números, hay muchísimos y crecen cada día.
- Cómo se mira son las **dimensiones**: por cliente, por producto, por fecha, por comercial. Son descripciones, hay relativamente pocas y cambian poco.

Una tabla de hechos en el centro, una tabla por dimensión alrededor, y cada dimensión unida al centro por una sola clave: dibujado, parece una estrella. La misma pregunta, en la estrella que vas a construir:

SQL · EN DUCKDB

```
SELECT c.provincia, p.categoria, round(sum(f.importe_neto)) AS ventas
FROM gold.fact_ventas AS f
JOIN gold.dim_cliente  AS c ON c.cliente_key  = f.cliente_key
JOIN gold.dim_producto AS p ON p.producto_key = f.producto_key
JOIN gold.dim_fecha    AS d ON d.fecha_key    = f.fecha_key
WHERE d.anio = 2025
GROUP BY c.provincia, p.categoria
ORDER BY ventas DESC
LIMIT 8;
```

RESULTADO

provincia varchar	categoria varchar	ventas double
Cantabria	Vinos	2395748.0
Cantabria	Cervezas	1948937.0
Cantabria	Alimentación	1425704.0
Cantabria	Refrescos	1282315.0
Asturias	Vinos	749135.0
Asturias	Cervezas	611474.0
Cantabria	Desechables	608634.0
Cantabria	Licores y destilados	583966.0

Las mismas cifras. Lo que ha desaparecido son las decisiones: los cancelados ya no están en `fact_ventas`, cada cliente ya es su ficha maestra, el año es una columna de la dimensión fecha y cada unión es "hechos con dimensión por su clave". No hay forma de unir mal, porque solo hay una forma de unir.

IDEA CLAVE

Silver tiene la forma del sistema que generó los datos. Gold tiene la forma de las preguntas que se hacen sobre ellos. Por eso gold se diseña empezando por las preguntas, no por las tablas de origen.

3. Hechos y dimensiones

3.1 Los hechos

Una **tabla de hechos** registra lo que pasa en un **proceso de negocio**: una venta, un pedido, un movimiento de almacén, una visita. Cada fila es un acontecimiento (o una foto de una situación, 3.4), y tiene dos tipos de columnas:

- **Claves** que apuntan a las dimensiones: `fecha_key`, `cliente_key`, `producto_key`...
- **Medidas**: los números que se suman, se cuentan o se promedian: `cantidad`, `importe_neto`.

Las tablas de hechos son largas y estrechas: muchas filas, pocas columnas, casi todas números. `fact_ventas` tendrá 366.928 filas y 14 columnas.

3.2 Las dimensiones

Una **dimensión** describe el contexto de los hechos. La forma rápida de encontrarlas es hacerse las preguntas de un periodista sobre cada venta:

PREGUNTA	DIMENSIÓN	ATRIBUTOS
¿Cuándo?	`dim_fecha`	Fecha, mes, nombre del mes, trimestre, año, día de la semana
¿Quién compra?	`dim_cliente`	Nombre, tipo de local, municipio, provincia, comercial asignado
¿Qué se vende?	`dim_producto`	SKU, nombre, categoría, marca, formato
¿Quién vende?	`dim_comercial`	Nombre, zona, puesto, jefe
¿Cómo?	(en la propia tabla de hechos, 4.5)	Canal, almacén, estado

Las dimensiones son al revés que los hechos: cortas y anchas. Pocas filas, muchas columnas descriptivas, casi todas de texto. Y las columnas de las dimensiones son exactamente lo que aparecerá en los filtros, los ejes y las leyendas de un informe de Power BI. Una buena dimensión tiene nombres que entiende alguien de negocio: `nombre_mes` = `marzo`, no `3`.

3.3 Tres tipos de medidas

No todos los números se pueden sumar, y confundirlo es uno de los errores más caros de un modelo:

TIPO	SE PUEDE SUMAR...	EJEMPLOS EN MAREJADA
Aditiva	Por cualquier dimensión	`importe_neto`, `cantidad`, `gastos_envio`
Semi-aditiva	Por algunas dimensiones, pero **no** por el tiempo	Las unidades de un inventario: se suman entre almacenes, pero no entre días (el stock del lunes más el del martes no es nada)
No aditiva	Por ninguna	`precio_unitario`, `descuento_pct`, cualquier porcentaje o media

Sumar una medida no aditiva no da error; da un número absurdo. Sumar los precios unitarios de las líneas de un mes, por ejemplo:

SQL · EN DUCKDB

```sql
SELECT round(sum(precio_unitario))      AS suma_de_precios,
       round(avg(precio_unitario), 2)  AS precio_medio_por_linea,
       round(sum(importe_neto) / sum(cantidad), 2) AS precio_medio_por_unidad
FROM gold.fact_ventas
WHERE fecha_key BETWEEN 20250301 AND 20250331;
```

RESULTADO

suma_de_precios decimal(38,0)	precio_medio_por_linea double	precio_medio_por_unidad double
235467	25.85	22.36

La suma de precios no significa nada. Y las dos medias son distintas, y las dos son "el precio medio": la primera trata igual una línea de una caja que una de cincuenta, la segunda pondera por unidades (y además es neta de descuentos). La regla: las medidas no aditivas no se guardan para sumarlas, sino que se **calculan** a partir de medidas aditivas en el momento de la consulta (`sum(importe) / sum(cantidad)`). En Power BI eso será una medida DAX (Volumen 08).

LA MEDIA DE LOS PORCENTAJES

El caso más frecuente de medida no aditiva es un porcentaje: el margen de cada producto, la tasa de cancelación de cada comercial. La media de los porcentajes no es el porcentaje del total, porque trata igual un producto que vende diez euros que uno que vende diez mil. El margen de una categoría es `1 - sum(coste) / sum(importe)`, nunca `avg(margen_producto)`.

3.4 Tres tipos de tablas de hechos

TIPO	CADA FILA ES...	EJEMPLO
Transaccional	Un acontecimiento, en el momento en que ocurre	Una línea de venta, un pedido
Foto periódica (*periodic snapshot*)	El estado de algo en un momento fijo, repetido cada día, semana o mes	El stock de cada producto en cada almacén cada día
Foto acumulativa (*accumulating snapshot*)	Un proceso con varias etapas, que se actualiza según avanza	Un pedido con su fecha de pedido, de preparación, de salida y de entrega

Las dos de Marejada son transaccionales. El inventario del 30 de septiembre sería una foto periódica con una sola foto; lo verás en el MJD-401.

3.5 Estrella y copo de nieve

En una estrella, cada dimensión es **una sola tabla**, aunque eso repita datos: la categoría "Cervezas" aparece escrita en las 84 filas de cervezas de `dim_producto`. Si en lugar de eso se normaliza la dimensión (una tabla `categorias` aparte, a la que apunta `productos`), el dibujo se ramifica y se llama **copo de nieve** (*snowflake*).

	ESTRELLA	COPO DE NIEVE
Dimensiones	Una tabla cada una, desnormalizada	Varias tablas por dimensión, normalizadas
Repetición de datos	Sí (la categoría en cada producto)	No

	ESTRELLA	COPO DE NIEVE
Uniones para consultar	Una por dimensión	Varias por dimensión
Para Power BI	Lo recomendado	Funciona, pero más lento y más difícil de usar

En un ERP, repetir "Cervezas" 84 veces sería un error de diseño. En un modelo analítico es lo correcto: las dimensiones son pequeñas, el espacio que se pierde es nada y a cambio cada pregunta necesita menos uniones. Es la diferencia de objetivo de la que hablaste en el Volumen 01 (4.1): el ERP escribe mucho y lee poco; el modelo analítico, al revés.

> **PARA EL PL-300**
>
> La documentación de Power BI recomienda expresamente el esquema en estrella, y el examen lo pregunta: cuándo desnormalizar una dimensión en copo de nieve (en Power Query, combinando las tablas), por qué las tablas de dimensión van en el lado "uno" de las relaciones y por qué conviene evitar las relaciones entre dos tablas de hechos. Todo eso sale de este capítulo.

3.6 La matriz de bus

Cuando hay varias tablas de hechos, un cuadro muy sencillo resume el modelo entero: procesos de negocio en las filas, dimensiones en las columnas, y una marca donde se usan. Kimball lo llamó **matriz de bus** (*bus matrix*):

PROCESO (TABLA DE HECHOS)	FECHA	CLIENTE	PRODUCTO	COMERCIAL	GRANO
Ventas (`fact_ventas`)	sí	sí	sí	sí	Una línea de pedido no cancelado
Pedidos (`fact_pedidos`)	sí (dos veces)	sí	—	sí	Un pedido

Lo importante es que las dos tablas de hechos **comparten** las mismas dimensiones de fecha, cliente y comercial. Por eso se pueden comparar: las ventas y los pedidos de un mismo cliente o de un mismo mes se agregan cada uno por su lado y se juntan por la dimensión común (11.4). A esas dimensiones compartidas se les llama **dimensiones conformadas**, y son lo que convierte varias estrellas sueltas en un modelo de empresa.

4. El grano

4.1 La primera decisión

En el Volumen 02 la pregunta era "¿qué es una fila en esta tabla?". En el modelado, esa pregunta se convierte en la **primera decisión de diseño**, antes de pensar en columnas: declarar el **grano** de cada tabla de hechos, en una frase. Para `fact_ventas`: "una fila es una línea de un pedido no cancelado".

Todo lo demás se deriva de esa frase. Las dimensiones que pueden ir en la tabla son las que tienen **un único valor** para cada fila de ese grano: una línea tiene un único producto, un único cliente (el de su pedido), una única fecha. Las medidas son las que tienen sentido para una línea: su cantidad, su importe.

4.2 El grano más fino posible

Podrías haber elegido otro grano: "las ventas de un cliente en un mes", por ejemplo, que tendría muchas menos filas. La recomendación casi universal es la contraria: **el grano más fino que permitan los datos**. La razón es que de lo fino se puede llegar a lo grueso agrupando, pero nunca al revés. Con una tabla por cliente y mes, nadie podría preguntar mañana por las ventas de un producto, ni por día de la semana.

4.3 Dos procesos, dos granos

¿Y los gastos de envío? Son de cada **pedido**, no de cada línea. Si los pusieras en `fact_ventas`, tendrías que repetirlos en todas las líneas del pedido o repartirlos de algún modo. Lo primero es la trampa del abanico del Volumen 02 (8.6), escrita en una tabla para siempre:

SQL · EN DUCKDB

```
SELECT pedido_id, v.num_linea, v.importe_neto, p.gastos_envio AS portes_si_los_pusieras_aqui
FROM gold.fact_ventas AS v
JOIN gold.fact_pedidos AS p USING (pedido_id)
WHERE pedido_id = 2025003295
ORDER BY v.num_linea;
```

RESULTADO

pedido_id	num_linea	importe_neto	portes_si_los_pusieras_aqui
int64	int64	double	decimal(10,2)
2025003295	1	31.15	12.00
2025003295	2	11.04	12.00
2025003295	3	18.2325	12.00

Doce euros de portes repetidos en cada línea: quien sume esa columna obtiene treinta y seis. Lo segundo (repartirlos entre las líneas, por ejemplo en proporción al importe) es posible, pero es una decisión de negocio que tendría que tomar Lucía, no tú.

La solución limpia es reconocer que son **dos procesos con dos granos**, y dos tablas de hechos: `fact_ventas` (grano línea) y `fact_pedidos` (grano pedido), con los portes en la segunda. Las dos comparten dimensiones, y cuando haga falta juntarlas se hará agregando cada una por su lado (11.4).

> **IDEA CLAVE**
>
> Una tabla de hechos, un grano. Si una medida no tiene un valor único para cada fila del grano, no pertenece a esa tabla: o pertenece a otra tabla de hechos con otro grano, o hay que decidir explícitamente cómo repartirla.

4.4 Qué entra y qué no

El grano también decide qué filas entran. `fact_ventas` deja fuera los pedidos cancelados: una línea cancelada no es una venta, y si estuviera dentro cada informe tendría que acordarse de filtrarla (que es justo lo que queremos evitar). Pero eso significa que con `fact_ventas` no se puede responder "¿cuántos pedidos se cancelan?". Esa pregunta va a `fact_pedidos`, que tiene **todos** los pedidos con su estado.

No hay una decisión correcta universal; hay una decisión **documentada**. Por eso la frase del grano dice "no cancelado", y por eso el capítulo 10 tiene una comprobación que cuenta las filas.

4.5 Las dimensiones degeneradas

Algunos atributos de los hechos no merecen una tabla propia: el número de pedido (`pedido_id`) no tiene nada que describir más allá de sí mismo, y el canal, el almacén o el estado son tres o cinco valores. Se dejan en la tabla de hechos como columnas de texto, y se llaman **dimensiones degeneradas**. `pedido_id` en `fact_ventas` es el ejemplo de manual: sirve para agrupar las líneas de un mismo pedido.

> **NOTA**
>
> Hay modelos que crean de todas formas una dimensión pequeña con el canal, el almacén y el estado (una *junk dimension*). En Power BI, con tres valores, da igual: puedes filtrar por una columna de la tabla de hechos sin problema. Lo veremos cuando montes el informe.

5. Las claves

5.1 Claves naturales y claves subrogadas

En silver, un cliente se identifica por su `cliente_id`, el número que le dio el ERP: una **clave natural** (viene del sistema de origen). En gold, cada dimensión tendrá además su propia clave, `cliente_key`, un número que genera el propio modelo y que no significa nada fuera de él: una **clave subrogada** (*surrogate key*). Las tablas de hechos apuntan a la subrogada.

Parece trabajo de más. Tiene cuatro razones:

RAZÓN	EN MAREJADA
Historial (SCD)	Un mismo cliente tendrá dos filas en `dim_cliente` (capítulo 8). Las dos tienen el mismo `cliente_id`; la clave subrogada es lo que las distingue
Independencia del origen	Si mañana Marejada cambia de ERP y renumera los clientes, el modelo no se rompe: solo cambia cómo se calcula la clave
Varios orígenes	La tienda web identifica a los clientes por NIF; el ERP, por número. Una clave propia permite unirlos sin depender de ninguno
El miembro desconocido	Los hechos que no encuentran su dimensión apuntan a una fila especial (5.2) en lugar de quedarse con la clave vacía

5.2 El miembro desconocido

Recuerda los 225 pedidos de clientes borrados del maestro y las 5 líneas con productos inexistentes. En silver, esos pedidos tienen `cliente_maestro_id` vacío. Si en gold la clave quedara vacía (`NULL`), pasarían dos cosas: un `INNER JOIN` con la dimensión los haría desaparecer de cualquier informe, y Power BI los agruparía bajo un "(En blanco)" que nadie entiende.

La solución estándar es una fila más en cada dimensión, con clave `-1` y descripciones que lo digan claro ("Cliente desconocido", "Producto desconocido"). Los hechos huérfanos apuntan a `-1`. Así **ningún hecho pierde su dimensión**, ninguna venta desaparece y en el informe aparece una línea que se puede explicar.

5.3 La clave de la fecha

La dimensión fecha es la excepción a "la clave subrogada no significa nada". Por convención, su clave es la fecha escrita como número: `20250314` para el 14 de marzo de 2025. Se llama a veces *smart key*, y tiene ventajas prácticas: es única, es fácil de calcular desde cualquier fecha sin consultar la dimensión, se lee a simple vista y filtrar un mes es un `BETWEEN 20250301 AND 20250331`.

5.4 Cómo se generan las claves

En este volumen las claves se generan con `row_number()` al construir cada dimensión. Funciona porque gold se reconstruye entera, dimensiones y hechos a la vez, desde silver: si una clave cambia de una ejecución a otra, los hechos se recalculan con la nueva.

EN PRODUCCIÓN, LAS CLAVES NO SE RECALCULAN

En un sistema real, las tablas de hechos son enormes y no se reconstruyen cada día: se les **añaden** las filas nuevas (carga incremental, Bloque 3). Entonces las claves subrogadas tienen que ser **estables**: un cliente conserva su `cliente_key` para siempre. Para eso los motores tienen columnas que generan números solos. En el Warehouse de Fabric es `IDENTITY` (solo admite `BIGINT`, y los valores son únicos pero no necesariamente consecutivos ni ordenados); en Databricks, `GENERATED ALWAYS AS IDENTITY` en las tablas Delta. Lo verás en sus bloques.

PARTE 2

Construir gold

Las seis tablas, una a una, con sus decisiones explicadas. Al final, un script que las construye todas y otro que las comprueba.

6. La dimensión fecha

7. Producto y comercial

8. Dimensiones que cambian: SCD

9. Las tablas de hechos

10. El script y sus comprobaciones

6. La dimensión fecha

6.1 Por qué una tabla de fechas

En el Volumen 03 (8.2) construiste un calendario con `generate_series` para no perder los meses sin ventas. En gold, ese calendario se convierte en una tabla permanente con todo lo que se puede preguntar de una fecha. Tiene tres ventajas sobre calcular `year(fecha)` o `month(fecha)` en cada consulta:

1. **Los nombres de negocio están escritos una vez**: `marzo`, `T1`, `lunes`. En español, y sin depender de la configuración de nadie.
2. **No faltan días**: hay una fila por cada día del periodo, haya ventas o no. Es lo que Power BI necesita para sus funciones de inteligencia de tiempo (12.3).
3. **Se pueden añadir atributos de negocio** que ninguna función calcula: festivos, temporada alta, semanas de campaña. Basta con una columna más.

6.2 Construirla

Es el primer bloque de `sql/03_construir_gold.sql`. Empieza por crear el esquema:

SQL · EN DUCKDB

```sql
CREATE SCHEMA IF NOT EXISTS gold;
```

SQL · EN DUCKDB

```sql
CREATE OR REPLACE TABLE gold.dim_fecha AS
SELECT CAST(strftime(d, '%Y%m%d') AS INTEGER)                    AS fecha_key,
       d::DATE                                                   AS fecha,
       year(d)                                                   AS anio,
       quarter(d)                                                AS trimestre,
       'T' || quarter(d)                                         AS nombre_trimestre,
       month(d)                                                  AS mes,
       ['enero', 'febrero', 'marzo', 'abril', 'mayo', 'junio', 'julio',
        'agosto', 'septiembre', 'octubre', 'noviembre', 'diciembre'][month(d)] AS nombre_mes,
       strftime(d, '%Y-%m')                                      AS anio_mes,
       date_trunc('month', d)::DATE                              AS inicio_mes,
       weekofyear(d)                                             AS semana_iso,
       day(d)                                                    AS dia_mes,
       isodow(d)                                                 AS dia_semana,
       ['lunes', 'martes', 'miércoles', 'jueves', 'viernes',
        'sábado', 'domingo'][isodow(d)]                          AS nombre_dia,
       isodow(d) >= 6                                            AS es_fin_de_semana
FROM generate_series(DATE '2024-01-01', DATE '2026-12-31', INTERVAL 1 DAY) AS t(d);
```

DESMONTANDO LA CONSULTA

`CREATE OR REPLACE TABLE gold.dim_fecha AS SELECT ...`

`CAST(strftime(d, '%Y%m%d') AS INTEGER)`	La clave: la fecha escrita como `20250314` (5.3). `strftime` la convierte en texto y `CAST` en número.
`['enero', ..., 'diciembre'][month(d)]`	Una lista de DuckDB y su elemento número `month(d)`. Las listas de DuckDB empiezan en 1, así que el mes 3 es `'marzo'`. Así el nombre del mes no depende del idioma del ordenador.
`nombre_trimestre`	El trimestre como texto (`'T'` unido al número del trimestre): `T1` … `T4`. Es lo que se verá en el eje de un gráfico.
`weekofyear(d)`	La semana del año según la norma ISO (la que se usa en Europa: empieza en lunes).
`DATE '2024-01-01' ... DATE '2026-12-31'`	**Años completos**, aunque los datos acaben en septiembre de 2026. Power BI lo pide así (12.3), y así una comparación de 2026 con 2025 tiene los mismos días disponibles.

SQL · EN DUCKDB

```
SELECT fecha_key, fecha, nombre_trimestre, nombre_mes, anio_mes, semana_iso, nombre_dia,
es_fin_de_semana
FROM gold.dim_fecha
WHERE fecha_key BETWEEN 20250310 AND 20250316
ORDER BY fecha_key;
```

RESULTADO

fecha_key int32	fecha date	nombre_trimestre varchar	…	semana_iso int64	nombre_dia varchar	es_fin_de_semana boolean
20250310	2025-03-10	T1	…	11	lunes	false
20250311	2025-03-11	T1	…	11	martes	false
20250312	2025-03-12	T1	…	11	miércoles	false
20250313	2025-03-13	T1	…	11	jueves	false
20250314	2025-03-14	T1	…	11	viernes	false
20250315	2025-03-15	T1	…	11	sábado	true
20250316	2025-03-16	T1	…	11	domingo	true

7 rows — 8 columns (6 shown)

SQL · EN DUCKDB

```
SELECT count(*) AS dias, min(fecha) AS desde, max(fecha) AS hasta,
       count(DISTINCT fecha) = count(*) AS fechas_unicas
FROM gold.dim_fecha;
```

RESULTADO

dias int64	desde date	hasta date	fechas_unicas boolean
1096	2024-01-01	2026-12-31	true

1.096 días: los 366 de 2024, que fue bisiesto, más los 365 de 2025 y los 365 de 2026.

FESTIVOS Y TEMPORADAS

Si Marcos te pidiera los plazos de entrega en días laborables, haría falta saber qué días son festivos en Cantabria y en Asturias. No hay función que lo sepa: se añade una columna `es_festivo` a `dim_fecha` a partir de una lista. Esa es la ventaja de tener la fecha como tabla: el negocio puede enriquecerla con lo que sabe.

7. Producto y comercial

7.1 dim_producto

Casi una copia de `silver.productos` con dos añadidos: la clave subrogada y el miembro desconocido.

SQL · EN DUCKDB

```
CREATE OR REPLACE TABLE gold.dim_producto AS
SELECT row_number() OVER (ORDER BY producto_id)::INTEGER AS producto_key,
       producto_id, sku, nombre, categoria, subcategoria, marca, formato,
       precio_lista, coste_unitario, iva_pct, activo
FROM silver.productos
UNION ALL
SELECT -1, NULL, 'DESCONOCIDO', 'Producto desconocido', 'Desconocida', 'Desconocida',
       'Desconocida', 'Desconocido', NULL, NULL, NULL, NULL;
```

DESMONTANDO LA CONSULTA

`SELECT row_number() OVER (ORDER BY producto_id)::INTEGER AS producto_key, ... UNION ALL SELECT -1, NULL, 'DESCONOCIDO', ...`

`row_number() OVER (ORDER BY producto_id)`	La clave subrogada: 1, 2, 3… por orden de `producto_id`. Una ventana del Volumen 03 usada para fabricar identificadores.
`::INTEGER`	`row_number()` devuelve un `BIGINT`; para una dimensión de 235 filas basta un entero normal.
`UNION ALL SELECT -1, ...`	El miembro desconocido (5.2). Tiene que tener **las mismas columnas en el mismo orden** que la primera parte; por eso los `NULL` en las que no tienen sentido.
`'Desconocida'`	En las columnas de texto que se usarán para agrupar (categoría, marca), un texto que diga lo que es. En las numéricas (precio, coste), `NULL`: un precio inventado sería peor que ninguno.

SQL · EN DUCKDB

```
SELECT producto_key, producto_id, sku, nombre, categoria, formato
FROM gold.dim_producto
WHERE producto_key IN (1, 2, 235, -1)
ORDER BY producto_key;
```

RESULTADO

producto_key int32	producto_id int64	sku varchar	nombre varchar	categoria varchar	formato varchar
-1	NULL	DESCONOCIDO	Producto desconocido	Desconocida	Desconocido
1	1001	CER-0001	Faro Norte Lager	Cervezas	Barril 30 L
2	1002	CER-0002	Faro Norte Lager	Cervezas	Barril 20 L
235	1235	DES-0008	Palillos redondos	Desechables	Caja 10000 uds

7.2 dim_comercial

SQL · EN DUCKDB

```
CREATE OR REPLACE TABLE gold.dim_comercial AS
SELECT row_number() OVER (ORDER BY c.comercial_id)::INTEGER AS comercial_key,
       c.comercial_id,
       c.nombre_completo           AS comercial,
       coalesce(c.zona, 'Dirección') AS zona,
       c.puesto,
       j.nombre_completo           AS jefe,
       c.fecha_alta,
       c.fecha_baja,
       c.fecha_baja IS NULL        AS en_activo
FROM silver.comerciales AS c
LEFT JOIN silver.comerciales AS j ON j.comercial_id = c.jefe_id
UNION ALL
SELECT -1, NULL, 'Desconocido', 'Desconocida', NULL, NULL, NULL, NULL, NULL;
```

El jefe se desnormaliza: en lugar de `jefe_id`, el **nombre** del jefe, que es lo que alguien querrá ver en un informe. Es el autojoin del Volumen 02 (8.5) guardado en la dimensión, para que nadie tenga que volver a escribirlo. Y la zona vacía de Íñigo se convierte en `Dirección`: en un filtro de Power BI, una zona en blanco no se entiende.

SQL · EN DUCKDB

```
SELECT comercial_key, comercial, zona, jefe
FROM gold.dim_comercial
ORDER BY comercial_key;
```

RESULTADO

comercial_key	comercial	zona	jefe
int32	varchar	varchar	varchar
-1	Desconocido	Desconocida	NULL
1	Íñigo Sainz Ruiz	Dirección	NULL
2	Raquel Fernández Cobo	Cantabria Centro	Íñigo Sainz Ruiz
3	Daniel Gutiérrez Pérez	Cantabria Centro	Raquel Fernández Cobo
4	Sara Ceballos Ortiz	Cantabria Centro	Raquel Fernández Cobo
5	Pablo Revuelta Gómez	Cantabria Oriental	Raquel Fernández Cobo
6	Lorena Arce Díaz	Cantabria Occidental	Raquel Fernández Cobo
7	Héctor Bustamante Lavín	Cantabria Occidental	Raquel Fernández Cobo
8	Nuria Álvarez Menéndez	Asturias	Íñigo Sainz Ruiz
9	Adrián Suárez Fernández	Asturias	Nuria Álvarez Menéndez
10	Ane Etxeberria Zubiri	Bizkaia	Nuria Álvarez Menéndez
11	Mikel Goikoetxea Arana	Bizkaia	Nuria Álvarez Menéndez
12	Rubén Martín Calvo	Castilla Norte	Íñigo Sainz Ruiz
13	Celia Gómez Sáiz	Cantabria Occidental	Raquel Fernández Cobo
14	Óscar Peña Revilla	Castilla Norte	Íñigo Sainz Ruiz

15 rows 4 columns

8. Dimensiones que cambian: SCD

8.1 El problema

Héctor Bustamante dejó Marejada el 30 de junio de 2025, y sus clientes pasaron a Celia Gómez. En silver, cada cliente tiene un solo `comercial_id`: el **actual**. Si construyeras `dim_cliente` con ese dato y Raquel preguntara "¿cuánto vendió en 2024 la cartera de cada comercial?", obtendría esto:

SQL · EN DUCKDB

```
SELECT year(p.fecha) AS anio,
       co.nombre_completo AS comercial_asignado_hoy,
       round(sum(l.importe_neto)) AS ventas
FROM silver.pedidos AS p
JOIN silver.lineas_pedido AS l ON l.pedido_id = p.pedido_id
JOIN silver.clientes AS c ON c.cliente_id = p.cliente_maestro_id
JOIN silver.comerciales AS co ON co.comercial_id = c.comercial_id
WHERE p.estado <> 'Cancelado'
  AND c.comercial_id IN (7, 13)
GROUP BY ALL
ORDER BY anio, comercial_asignado_hoy;
```

RESULTADO

```
┌────────┬──────────────────────┬──────────┐
│ anio   │ comercial_asignado_hoy │  ventas  │
│ int64  │        varchar         │  double  │
├────────┼──────────────────────┼──────────┤
│  2024  │ Celia Gómez Sáiz       │ 1047773.0 │
│  2025  │ Celia Gómez Sáiz       │ 1307724.0 │
│  2026  │ Celia Gómez Sáiz       │ 1063105.0 │
└────────┴──────────────────────┴──────────┘
```

Según esto, la cartera de Celia vendió más de un millón de euros en 2024, un año antes de que ella entrara en la empresa, y la de Héctor no vendió nunca nada. Es la historia reescrita: al cambiar el comercial asignado, **todo el pasado** del cliente se ha ido con él.

Este problema tiene nombre: una **dimensión lentamente cambiante** (*slowly changing dimension*, **SCD**). Los atributos de una dimensión cambian de vez en cuando (un cliente cambia de comercial, de dirección, de tipo de local) y hay que decidir qué pasa con la historia.

8.2 Los tipos de SCD

TIPO	QUÉ HACE AL CAMBIAR EL ATRIBUTO	CONSECUENCIA	CUÁNDO
0	Nada: el valor original no cambia nunca	El pasado y el presente usan el valor inicial	Atributos que no deben cambiar: la fecha de alta
1	Sobrescribe el valor	Todo, pasado incluido, pasa a usar el valor nuevo	Correcciones (una errata en el nombre) o cuando la historia no importa
2	Añade una **fila nueva** con el valor nuevo y cierra la anterior	Cada hecho conserva el valor que tenía cuando ocurrió	Cuando la historia importa: el comercial asignado, la zona, la tarifa
3	Añade una **columna** con el valor anterior	Se ve el valor actual y el inmediatamente anterior, nada más	Pocas veces; cambios únicos y planificados

PARA EL DP-600 Y EL DE DATABRICKS

Los tipos 1 y 2 aparecen en los dos exámenes, sobre todo el 2: reconocer en un enunciado que hay que conservar el historial, saber qué columnas necesita (clave subrogada, fechas de validez, marca de versión actual) y cómo se implementa con `MERGE`. Tipo 1 = sobrescribir; tipo 2 = nueva fila. Con eso aciertas la mayoría.

8.3 SCD tipo 2 sobre el comercial asignado

En `dim_cliente`, el comercial asignado será de tipo 2; el resto de atributos, de tipo 1 (si un cliente cambia de nombre comercial, se ve el nombre nuevo en todo su historial, que es lo que se quiere). Cada cliente tendrá una fila por cada periodo con un comercial distinto, con tres columnas nuevas:

COLUMNA	QUÉ GUARDA
`valido_desde`	Primer día en que esta versión es cierta
`valido_hasta`	Último día en que es cierta. Para la versión actual, una fecha muy lejana: `9999-12-31`
`es_version_actual`	`true` solo en la versión vigente. Es lo que se filtra para ver "los clientes tal como están hoy"

El ERP no guarda el historial de asignaciones, así que hay que reconstruirlo con lo que sabemos: RRHH registra dos salidas (Óscar Peña el 31 de octubre de 2024, cuyos clientes pasaron a Rubén Martín; y Héctor el 30 de junio de 2025, cuyos clientes pasaron a Celia), y los pedidos dicen qué clientes llevó cada uno. La regla: un cliente que hoy es de Rubén o de Celia "era de" Óscar o de Héctor si tiene algún pedido gestionado por esa persona.

SQL · EN DUCKDB

```
CREATE OR REPLACE TABLE gold.dim_cliente AS
WITH maestros AS (
    SELECT * FROM silver.clientes WHERE NOT es_ficha_duplicada
),
reasignaciones AS (
    SELECT * FROM (VALUES
        (14, 12, DATE '2024-10-31'),
        ( 7, 13, DATE '2025-06-30')
    ) AS t(comercial_anterior, comercial_nuevo, ultimo_dia_anterior)
),
heredados AS (
    SELECT DISTINCT m.cliente_id, r.comercial_anterior, r.ultimo_dia_anterior
    FROM maestros AS m
    JOIN reasignaciones AS r ON r.comercial_nuevo = m.comercial_id
    JOIN silver.pedidos AS p
      ON p.cliente_maestro_id = m.cliente_id
     AND p.comercial_id = r.comercial_anterior
),
versiones AS (
    -- versión anterior: solo para los clientes heredados
    SELECT m.*, h.comercial_anterior AS comercial_asignado_id,
           DATE '1900-01-01' AS valido_desde, h.ultimo_dia_anterior AS valido_hasta
    FROM maestros AS m
    JOIN heredados AS h USING (cliente_id)
    UNION ALL
    -- versión actual: todos los clientes
    SELECT m.*, m.comercial_id AS comercial_asignado_id,
           coalesce(h.ultimo_dia_anterior + 1, DATE '1900-01-01') AS valido_desde,
           DATE '9999-12-31' AS valido_hasta
    FROM maestros AS m
    LEFT JOIN heredados AS h USING (cliente_id)
)
SELECT row_number() OVER (ORDER BY v.cliente_id, v.valido_desde)::INTEGER AS cliente_key,
       v.cliente_id,
       v.nombre_comercial,
       v.razon_social,
       v.tipo_cliente,
       v.municipio,
       v.provincia,
       v.codigo_postal,
       v.comercial_asignado_id,
       co.nombre_completo                AS comercial_asignado,
       v.fecha_alta,
       v.activo_en_erp,
       v.valido_desde,
       v.valido_hasta,
       v.valido_hasta = DATE '9999-12-31' AS es_version_actual
FROM versiones AS v
LEFT JOIN silver.comerciales AS co ON co.comercial_id = v.comercial_asignado_id
UNION ALL
SELECT -1, NULL, 'Cliente desconocido', NULL, 'Desconocido', NULL, 'Desconocida', NULL,
       NULL, NULL, NULL, NULL, DATE '1900-01-01', DATE '9999-12-31', true;
```

DESMONTANDO EL COMANDO

maestros · reasignaciones · heredados · versiones

maestros	Solo las fichas maestras: un cliente real, una fila (por ahora). Las fichas duplicadas no existen en gold.
reasignaciones	Lo que sabemos por RRHH, escrito como una tabla al vuelo (`VALUES`): quién se fue, quién heredó sus clientes y su último día.
heredados	Los clientes del heredero que tienen pedidos gestionados por quien se fue. `DISTINCT` porque un cliente tiene muchos pedidos.
versiones · primera parte	Para los heredados, la versión antigua: el comercial que se fue, válida desde siempre (`1900-01-01`) hasta su último día.
versiones · segunda parte	Para todos, la versión actual: el comercial de hoy, válida desde el día siguiente a la reasignación (o desde siempre, si no la hubo) hasta `9999-12-31`.
row_number() OVER (ORDER BY cliente_id, valido_desde)	La clave subrogada: distinta para cada versión. Ahora sí es imprescindible (5.1).

SQL · EN DUCKDB

```
SELECT count(*) AS filas,
       count(DISTINCT cliente_id) AS clientes,
       count(*) FILTER (WHERE NOT es_version_actual) AS versiones_antiguas
FROM gold.dim_cliente
WHERE cliente_key <> -1;
```

RESULTADO

```
┌───────┬──────────┬────────────────────┐
│ filas │ clientes │ versiones_antiguas │
│ int64 │  int64   │       int64        │
├───────┼──────────┼────────────────────┤
│  1348 │     1175 │                173 │
└───────┴──────────┴────────────────────┘
```

1.175 clientes reales y 173 versiones antiguas: 83 clientes heredados de Óscar y 90 de Héctor. Mira uno:

SQL · EN DUCKDB

```
SELECT cliente_key, cliente_id, comercial_asignado,
       valido_desde, valido_hasta, es_version_actual
FROM gold.dim_cliente
WHERE cliente_id = (SELECT min(cliente_id) FROM gold.dim_cliente
                    WHERE comercial_asignado_id = 7)
ORDER BY valido_desde;
```

RESULTADO

```
┌─────────────┬────────────┬──────────────────────┬───┬──────────────┬──────────────────┐
│ cliente_key │ cliente_id │  comercial_asignado  │ … │ valido_hasta │ es_version_actual │
│    int32    │   int64    │       varchar        │ … │     date     │      boolean      │
├─────────────┼────────────┼──────────────────────┼───┼──────────────┼──────────────────┤
│          12 │      10011 │ Héctor Bustamante Lavín │ … │ 2025-06-30   │ false            │
│          13 │      10011 │ Celia Gómez Sáiz     │ … │ 9999-12-31   │ true             │
└─────────────┴────────────┴──────────────────────┴───┴──────────────┴──────────────────┘
  2 rows                                                    6 columns (5 shown)
```

Dos filas, el mismo cliente, dos claves. Hasta el 30 de junio de 2025, de Héctor; desde el 1 de julio, de Celia. Y ahora la pregunta de Raquel, con la dimensión de tipo 2 (los hechos ya apuntan a la versión vigente en la fecha de cada pedido; lo construyes en el capítulo 9):

SQL · EN DUCKDB

```
SELECT d.anio, c.comercial_asignado, round(sum(f.importe_neto)) AS ventas
FROM gold.fact_ventas AS f
JOIN gold.dim_cliente AS c ON c.cliente_key = f.cliente_key
JOIN gold.dim_fecha   AS d ON d.fecha_key   = f.fecha_key
WHERE c.comercial_asignado_id IN (7, 13)
GROUP BY ALL
ORDER BY d.anio, c.comercial_asignado;
```

RESULTADO

```
┌───────┬──────────────────────┬─────────┐
│ anio  │  comercial_asignado  │ ventas  │
│ int64 │       varchar        │ double  │
├───────┼──────────────────────┼─────────┤
│  2024 │ Héctor Bustamante Lavín │ 1047773.0 │
│  2025 │ Celia Gómez Sáiz     │  711309.0 │
│  2025 │ Héctor Bustamante Lavín │  596416.0 │
│  2026 │ Celia Gómez Sáiz     │ 1063105.0 │
└───────┴──────────────────────┴─────────┘
```

Ahora 2024 es de Héctor, 2025 se reparte entre los dos el 1 de julio, y 2026 es de Celia. La historia está bien contada.

CONTAR CLIENTES CON UNA DIMENSIÓN DE TIPO 2

Con historial, `count(*)` sobre `dim_cliente` ya no cuenta clientes, sino versiones: 1.348 en lugar de 1.175. Para contar clientes, `count(DISTINCT cliente_id)`, o filtrar `es_version_actual`. Es el error más común al empezar a usar una SCD de tipo 2, y aparece igual en Power BI.

8.4 La comprobación de una SCD de tipo 2

Lo que nunca puede pasar en una dimensión de tipo 2 es que dos versiones del mismo cliente se solapen: un pedido de ese día encontraría dos filas y se duplicaría (el abanico, otra vez). La comprobación busca parejas de versiones del mismo cliente cuyos periodos se cruzan:

SQL · EN DUCKDB

```
SELECT count(*) AS versiones_solapadas
FROM gold.dim_cliente AS a
JOIN gold.dim_cliente AS b
  ON a.cliente_id = b.cliente_id
 AND a.cliente_key < b.cliente_key
 AND a.valido_desde <= b.valido_hasta
 AND b.valido_desde <= a.valido_hasta;
```

RESULTADO

```
┌────────────────────┐
│ versiones_solapadas │
│       int64        │
├────────────────────┤
│                  0 │
└────────────────────┘
```

DESMONTANDO EL COMANDO

`a.valido_desde <= b.valido_hasta AND b.valido_desde <= a.valido_hasta`

`a.cliente_key < b.cliente_key`	Cada pareja una sola vez, y nunca una versión consigo misma.
`a.valido_desde <= b.valido_hasta`	La versión `a` empieza antes de que termine la `b`...
`AND b.valido_desde <= a.valido_hasta`	...y la `b` empieza antes de que termine la `a`. Si las dos se cumplen, los periodos se cruzan. Es la forma estándar de comprobar si dos intervalos se solapan, y te la preguntarán.

Cero. Va a la lista de comprobaciones del capítulo 10.

9. Las tablas de hechos

9.1 fact_ventas

La tabla de hechos se construye desde silver, sustituyendo cada clave natural por la clave subrogada de su dimensión. Esa operación se llama **búsqueda de claves** (*key lookup*), y es un `LEFT JOIN` por dimensión:

SQL · EN DUCKDB

```
CREATE OR REPLACE TABLE gold.fact_ventas AS
SELECT l.pedido_id,
       l.num_linea,
       CAST(strftime(p.fecha, '%Y%m%d') AS INTEGER)  AS fecha_key,
       coalesce(dc.cliente_key, -1)                  AS cliente_key,
       coalesce(dp.producto_key, -1)                 AS producto_key,
       coalesce(dco.comercial_key, -1)               AS comercial_key,
       p.canal,
       p.almacen,
       p.estado,
       l.cantidad,
       l.precio_unitario,
       l.descuento_pct,
       l.importe_neto,
       l.cantidad * dp.coste_unitario                AS coste_estimado
FROM silver.lineas_pedido AS l
JOIN silver.pedidos AS p ON p.pedido_id = l.pedido_id
LEFT JOIN gold.dim_cliente AS dc
       ON dc.cliente_id = p.cliente_maestro_id
      AND p.fecha BETWEEN dc.valido_desde AND dc.valido_hasta
LEFT JOIN gold.dim_producto AS dp ON dp.producto_id = l.producto_id
LEFT JOIN gold.dim_comercial AS dco ON dco.comercial_id = p.comercial_id
WHERE p.estado <> 'Cancelado';
```

DESMONTANDO LA CONSULTA

`LEFT JOIN gold.dim_cliente AS dc ON dc.cliente_id = p.cliente_maestro_id AND p.fecha BETWEEN dc.valido_desde AND dc.valido_hasta`

`dc.cliente_id = p.cliente_maestro_id`	Por la ficha maestra: así las ventas de las fichas duplicadas van a su cliente real.
`AND p.fecha BETWEEN dc.valido_desde AND dc.valido_hasta`	De todas las versiones del cliente, la que estaba vigente el día del pedido. Es lo que hace que el tipo 2 funcione: el pedido de 2024 apunta a la versión de Héctor.
`LEFT JOIN`	Para que una venta sin cliente en el maestro no desaparezca.
`coalesce(dc.cliente_key, -1)`	Y esa venta apunta al miembro desconocido.
`CAST(strftime(p.fecha, '%Y%m%d') AS INTEGER)`	La clave de fecha se calcula sin buscar en `dim_fecha`: es la ventaja de la *smart key* (5.3).
`l.cantidad * dp.coste_unitario AS coste_estimado`	El coste de la línea. Es una estimación: 9.3.
`WHERE p.estado <> 'Cancelado'`	El grano: líneas de pedidos no cancelados (4.4).

¿Cuántas ventas apuntan a los miembros desconocidos?

SQL · EN DUCKDB

```
SELECT count(*) FILTER (WHERE cliente_key = -1)                       AS lineas_sin_cliente,
       round(sum(importe_neto) FILTER (WHERE cliente_key = -1), 2)    AS importe_sin_cliente,
       count(*) FILTER (WHERE producto_key = -1)                      AS lineas_sin_producto,
       round(sum(importe_neto) FILTER (WHERE producto_key = -1), 2)   AS importe_sin_producto
FROM gold.fact_ventas;
```

RESULTADO

lineas_sin_cliente	importe_sin_cliente	lineas_sin_producto	importe_sin_producto
int64	double	int64	double
1056	125777.83	5	323.75

Con un `INNER JOIN` en la construcción, esos 125.777,83 euros de clientes borrados no estarían en el modelo, y ningún informe lo habría avisado.

9.2 fact_pedidos

SQL · EN DUCKDB

```
CREATE OR REPLACE TABLE gold.fact_pedidos AS
WITH lineas AS (
    SELECT pedido_id, count(*) AS num_lineas, sum(importe_neto) AS importe_neto
    FROM silver.lineas_pedido
    GROUP BY pedido_id
)
SELECT p.pedido_id,
       CAST(strftime(p.fecha, '%Y%m%d') AS INTEGER)          AS fecha_key,
       CAST(strftime(p.fecha_entrega, '%Y%m%d') AS INTEGER)  AS fecha_entrega_key,
       coalesce(dc.cliente_key, -1)                          AS cliente_key,
       coalesce(dco.comercial_key, -1)                       AS comercial_key,
       p.canal,
       p.almacen,
       p.estado,
       p.entrega_sospechosa,
       coalesce(l.num_lineas, 0)                             AS num_lineas,
       coalesce(l.importe_neto, 0)                           AS importe_neto,
       p.gastos_envio,
       p.fecha_entrega - p.fecha                             AS dias_entrega
FROM silver.pedidos AS p
LEFT JOIN lineas AS l ON l.pedido_id = p.pedido_id
LEFT JOIN gold.dim_cliente AS dc
       ON dc.cliente_id = p.cliente_maestro_id
      AND p.fecha BETWEEN dc.valido_desde AND dc.valido_hasta
LEFT JOIN gold.dim_comercial AS dco ON dco.comercial_id = p.comercial_id;
```

Tres detalles que la distinguen de `fact_ventas`:

- **Todos los pedidos**, con su estado: aquí se cuentan los cancelados, los pendientes y los plazos de entrega.
- **Dos fechas**: la del pedido y la de entrega, las dos como claves de `dim_fecha`. Es una dimensión que juega dos papeles (11.3).
- **La fecha de entrega puede estar vacía** (pedidos sin entregar y las 23 imposibles de silver). Aquí no se usa un miembro desconocido, por una razón práctica: Power BI exige que la tabla de fechas no tenga filas en blanco ni repetidas para poder marcarla como tabla de fechas (12.3), y una fila "sin fecha" lo rompería.

SQL · EN DUCKDB

```
SELECT estado, count(*) AS pedidos,
       count(fecha_entrega_key) AS con_fecha_entrega,
       round(avg(dias_entrega), 2) AS dias_medios
FROM gold.fact_pedidos
GROUP BY estado
ORDER BY pedidos DESC;
```

RESULTADO

estado varchar	pedidos int64	con_fecha_entrega int64	dias_medios double
Entregado	73280	73257	2.15
Cancelado	2732	0	NULL
Devuelto parcial	1164	1164	2.15
En reparto	140	0	NULL
Pendiente	132	0	NULL

Las cifras de control del máster: 73.280 entregados, 2.732 cancelados, 1.164 devueltos parcialmente, 140 en reparto y 132 pendientes, 77.448 en total. Y 23 pedidos entregados sin fecha de entrega: las fechas imposibles que silver dejó vacías.

9.3 Una medida con un supuesto

`coste_estimado` multiplica la cantidad de cada línea por el `coste_unitario` del maestro de productos. Pero ese coste es el **de hoy**: el ERP no guarda cuánto costaba cada producto en 2024. Para 2026 la estimación es buena; para 2024 subestima el margen, porque los precios de venta de 2024 eran más bajos y el coste que se le aplica es el de 2026.

No es un error tuyo, es una limitación de los datos. Lo que sí sería un error es no decirlo. Por eso la columna se llama `coste_estimado` y no `coste`, y por eso el MJD-404 te pide escribirlo en el entregable. Es, por cierto, otro caso de dimensión que cambia: el coste de un producto es un atributo que el ERP sobrescribe (tipo 1) cuando el análisis de márgenes necesitaría el historial (tipo 2).

10. El script y sus comprobaciones

10.1 Todo junto

Lo que has ejecutado en los capítulos 6 a 9 va, en ese orden, a `sql/03_construir_gold.sql`. El orden importa por dos motivos: las dimensiones van antes que los hechos (los hechos buscan sus claves en ellas), y `dim_cliente` usa `silver.comerciales`, que ya existe. El fichero completo:

FICHERO · sql/03_construir_gold.sql

```
-- ============================================================
-- Capa GOLD de Marejada Distribución: el modelo en estrella
-- Se construye a partir de silver. Volumen 04.
-- Uso:  duckdb db/marejada.duckdb < sql/03_construir_gold.sql
--       duckdb db/marejada.duckdb < sql/04_comprobar_gold.sql
-- ============================================================

CREATE SCHEMA IF NOT EXISTS gold;

-- ------------------------------------------------------------
-- dim_fecha: un día por fila, años completos 2024-2026
-- ------------------------------------------------------------
CREATE OR REPLACE TABLE gold.dim_fecha AS
SELECT CAST(strftime(d, '%Y%m%d') AS INTEGER)                    AS fecha_key,
       d::DATE                                                   AS fecha,
       year(d)                                                   AS anio,
       quarter(d)                                                AS trimestre,
       'T' || quarter(d)                                         AS nombre_trimestre,
       month(d)                                                  AS mes,
       ['enero', 'febrero', 'marzo', 'abril', 'mayo', 'junio', 'julio',
        'agosto', 'septiembre', 'octubre', 'noviembre', 'diciembre'][month(d)] AS nombre_mes,
       strftime(d, '%Y-%m')                                      AS anio_mes,
       date_trunc('month', d)::DATE                              AS inicio_mes,
       weekofyear(d)                                             AS semana_iso,
       day(d)                                                    AS dia_mes,
       isodow(d)                                                 AS dia_semana,
       ['lunes', 'martes', 'miércoles', 'jueves', 'viernes',
        'sábado', 'domingo'][isodow(d)]                          AS nombre_dia,
       isodow(d) >= 6                                            AS es_fin_de_semana
FROM generate_series(DATE '2024-01-01', DATE '2026-12-31', INTERVAL 1 DAY) AS t(d);

-- ------------------------------------------------------------
-- dim_producto: un producto por fila + el miembro desconocido
-- ------------------------------------------------------------
CREATE OR REPLACE TABLE gold.dim_producto AS
SELECT row_number() OVER (ORDER BY producto_id)::INTEGER AS producto_key,
       producto_id, sku, nombre, categoria, subcategoria, marca, formato,
       precio_lista, coste_unitario, iva_pct, activo
FROM silver.productos
UNION ALL
SELECT -1, NULL, 'DESCONOCIDO', 'Producto desconocido', 'Desconocida', 'Desconocida',
       'Desconocida', 'Desconocido', NULL, NULL, NULL, NULL;

-- ------------------------------------------------------------
-- dim_comercial: una persona por fila + el miembro desconocido
-- ------------------------------------------------------------
CREATE OR REPLACE TABLE gold.dim_comercial AS
SELECT row_number() OVER (ORDER BY c.comercial_id)::INTEGER AS comercial_key,
       c.comercial_id,
       c.nombre_completo          AS comercial,
       coalesce(c.zona, 'Dirección') AS zona,
       c.puesto,
       j.nombre_completo          AS jefe,
       c.fecha_alta,
       c.fecha_baja,
       c.fecha_baja IS NULL       AS en_activo
FROM silver.comerciales AS c
LEFT JOIN silver.comerciales AS j ON j.comercial_id = c.jefe_id
UNION ALL
SELECT -1, NULL, 'Desconocido', 'Desconocida', NULL, NULL, NULL, NULL, NULL;

-- ------------------------------------------------------------
-- dim_cliente: SCD tipo 2 sobre el comercial asignado.
-- Una fila por cliente real (ficha maestra) y por cada periodo
-- con un comercial asignado distinto.
-- Reasignaciones conocidas (RRHH):
--   Óscar Peña (14) se fue el 31/10/2024: sus clientes pasan a Rubén Martín (12).
--   Héctor Bustamante (7) se fue el 30/06/2025: sus clientes pasan a Celia Gómez (13).
-- Un cliente "era de" quien se fue si tiene algún pedido gestionado por esa persona.
```

```
-- ------------------------------------------------------------
CREATE OR REPLACE TABLE gold.dim_cliente AS
WITH maestros AS (
    SELECT * FROM silver.clientes WHERE NOT es_ficha_duplicada
),
reasignaciones AS (
    SELECT * FROM (VALUES
        (14, 12, DATE '2024-10-31'),
        ( 7, 13, DATE '2025-06-30')
    ) AS t(comercial_anterior, comercial_nuevo, ultimo_dia_anterior)
),
heredados AS (
    SELECT DISTINCT m.cliente_id, r.comercial_anterior, r.ultimo_dia_anterior
    FROM maestros AS m
    JOIN reasignaciones AS r ON r.comercial_nuevo = m.comercial_id
    JOIN silver.pedidos AS p
      ON p.cliente_maestro_id = m.cliente_id
     AND p.comercial_id = r.comercial_anterior
),
versiones AS (
    -- versión anterior: solo para los clientes heredados
    SELECT m.*, h.comercial_anterior AS comercial_asignado_id,
           DATE '1900-01-01' AS valido_desde, h.ultimo_dia_anterior AS valido_hasta
    FROM maestros AS m
    JOIN heredados AS h USING (cliente_id)
    UNION ALL
    -- versión actual: todos los clientes
    SELECT m.*, m.comercial_id AS comercial_asignado_id,
           coalesce(h.ultimo_dia_anterior + 1, DATE '1900-01-01') AS valido_desde,
           DATE '9999-12-31' AS valido_hasta
    FROM maestros AS m
    LEFT JOIN heredados AS h USING (cliente_id)
)
SELECT row_number() OVER (ORDER BY v.cliente_id, v.valido_desde)::INTEGER AS cliente_key,
       v.cliente_id,
       v.nombre_comercial,
       v.razon_social,
       v.tipo_cliente,
       v.municipio,
       v.provincia,
       v.codigo_postal,
       v.comercial_asignado_id,
       co.nombre_completo                AS comercial_asignado,
       v.fecha_alta,
       v.activo_en_erp,
       v.valido_desde,
       v.valido_hasta,
       v.valido_hasta = DATE '9999-12-31' AS es_version_actual
FROM versiones AS v
LEFT JOIN silver.comerciales AS co ON co.comercial_id = v.comercial_asignado_id
UNION ALL
SELECT -1, NULL, 'Cliente desconocido', NULL, 'Desconocido', NULL, 'Desconocida', NULL,
       NULL, NULL, NULL, NULL, DATE '1900-01-01', DATE '9999-12-31', true;

-- ------------------------------------------------------------
-- fact_ventas. Grano: una línea de un pedido no cancelado.
-- ------------------------------------------------------------
CREATE OR REPLACE TABLE gold.fact_ventas AS
SELECT l.pedido_id,
       l.num_linea,
       CAST(strftime(p.fecha, '%Y%m%d') AS INTEGER)  AS fecha_key,
       coalesce(dc.cliente_key, -1)                  AS cliente_key,
       coalesce(dp.producto_key, -1)                 AS producto_key,
       coalesce(dco.comercial_key, -1)               AS comercial_key,
       p.canal,
       p.almacen,
       p.estado,
       l.cantidad,
       l.precio_unitario,
       l.descuento_pct,
```

```
       l.importe_neto,
       l.cantidad * dp.coste_unitario               AS coste_estimado
FROM silver.lineas_pedido AS l
JOIN silver.pedidos AS p ON p.pedido_id = l.pedido_id
LEFT JOIN gold.dim_cliente AS dc
       ON dc.cliente_id = p.cliente_maestro_id
      AND p.fecha BETWEEN dc.valido_desde AND dc.valido_hasta
LEFT JOIN gold.dim_producto AS dp ON dp.producto_id = l.producto_id
LEFT JOIN gold.dim_comercial AS dco ON dco.comercial_id = p.comercial_id
WHERE p.estado <> 'Cancelado';

-- ------------------------------------------------------------
-- fact_pedidos. Grano: un pedido, en cualquier estado.
-- Aquí viven los gastos de envío, que son del pedido, no de la línea.
-- ------------------------------------------------------------
CREATE OR REPLACE TABLE gold.fact_pedidos AS
WITH lineas AS (
    SELECT pedido_id, count(*) AS num_lineas, sum(importe_neto) AS importe_neto
    FROM silver.lineas_pedido
    GROUP BY pedido_id
)
SELECT p.pedido_id,
       CAST(strftime(p.fecha, '%Y%m%d') AS INTEGER)          AS fecha_key,
       CAST(strftime(p.fecha_entrega, '%Y%m%d') AS INTEGER)  AS fecha_entrega_key,
       coalesce(dc.cliente_key, -1)                          AS cliente_key,
       coalesce(dco.comercial_key, -1)                       AS comercial_key,
       p.canal,
       p.almacen,
       p.estado,
       p.entrega_sospechosa,
       coalesce(l.num_lineas, 0)                             AS num_lineas,
       coalesce(l.importe_neto, 0)                           AS importe_neto,
       p.gastos_envio,
       p.fecha_entrega - p.fecha                             AS dias_entrega
FROM silver.pedidos AS p
LEFT JOIN lineas AS l ON l.pedido_id = p.pedido_id
LEFT JOIN gold.dim_cliente AS dc
       ON dc.cliente_id = p.cliente_maestro_id
      AND p.fecha BETWEEN dc.valido_desde AND dc.valido_hasta
LEFT JOIN gold.dim_comercial AS dco ON dco.comercial_id = p.comercial_id;
```

10.2 Las comprobaciones

Y `sql/04_comprobar_gold.sql`, con el mismo formato que el de silver: una fila por comprobación y una columna `ok`. Cada comprobación sale de una decisión de este volumen.

SQL · EN DUCKDB

```
-- ============================================================
-- Comprobaciones de la capa GOLD. Todas deben salir true.
-- Uso:  duckdb db/marejada.duckdb < sql/04_comprobar_gold.sql
-- ============================================================
SELECT 'fact_ventas: mismas líneas que silver sin cancelados' AS comprobacion,
       (SELECT count(*) FROM gold.fact_ventas)
     = (SELECT count(*) FROM silver.lineas_pedido AS l
        JOIN silver.pedidos AS p ON p.pedido_id = l.pedido_id
        WHERE p.estado <> 'Cancelado') AS ok
UNION ALL
SELECT 'fact_ventas: mismo importe que silver sin cancelados',
       abs((SELECT sum(importe_neto) FROM gold.fact_ventas)
         - (SELECT sum(l.importe_neto) FROM silver.lineas_pedido AS l
            JOIN silver.pedidos AS p ON p.pedido_id = l.pedido_id
            WHERE p.estado <> 'Cancelado')) < 0.01
UNION ALL
SELECT 'fact_pedidos: un pedido por fila, todos los de silver',
       (SELECT count(*) FROM gold.fact_pedidos) = (SELECT count(*) FROM silver.pedidos)
   AND (SELECT count(DISTINCT pedido_id) FROM gold.fact_pedidos) = (SELECT count(*) FROM
silver.pedidos)
UNION ALL
SELECT 'Hechos: ninguna clave de dimensión vacía',
       (SELECT count(*) FROM gold.fact_ventas
        WHERE fecha_key IS NULL OR cliente_key IS NULL OR producto_key IS NULL OR comercial_key IS
NULL) = 0
UNION ALL
SELECT 'Hechos: todas las fechas existen en dim_fecha',
       (SELECT count(*) FROM gold.fact_pedidos AS f
        LEFT JOIN gold.dim_fecha AS d ON d.fecha_key = f.fecha_key
        WHERE d.fecha_key IS NULL) = 0
UNION ALL
SELECT 'dim_cliente: una sola versión válida por cliente y día',
       (SELECT count(*) FROM gold.dim_cliente AS a
        JOIN gold.dim_cliente AS b
          ON a.cliente_id = b.cliente_id AND a.cliente_key < b.cliente_key
         AND a.valido_desde <= b.valido_hasta AND b.valido_desde <= a.valido_hasta) = 0
UNION ALL
SELECT 'Puente de marzo de 2025: 1.082.243,50 como contabilidad',
       (SELECT round(sum(importe_neto + gastos_envio), 2)
        FROM gold.fact_pedidos
        WHERE estado IN ('Entregado', 'Devuelto parcial')
          AND fecha_entrega_key BETWEEN 20250301 AND 20250331) = 1082243.50;
```

RESULTADO

comprobacion varchar	ok boolean
fact_ventas: mismas líneas que silver sin cancelados	true
fact_ventas: mismo importe que silver sin cancelados	true
fact_pedidos: un pedido por fila, todos los de silver	true
Hechos: ninguna clave de dimensión vacía	true
Hechos: todas las fechas existen en dim_fecha	true
dim_cliente: una sola versión válida por cliente y día	true
Puente de marzo de 2025: 1.082.243,50 como contabilidad	true

Siete `true`. La última es la más importante: el modelo reproduce **al céntimo** la cifra de contabilidad de marzo de 2025 que te costó un ticket entero en el Volumen 02. Si un día alguien cambia algo en silver o en gold y esa cifra se mueve, lo sabrás antes que Lucía.

10.3 Reconstruir todo desde cero

Cuatro scripts, de los ficheros originales al modelo comprobado:

TÚ ESCRIBES

```
rm db/marejada.duckdb
time (duckdb db/marejada.duckdb < sql/00_cargar_bronze.sql && \
      duckdb db/marejada.duckdb < sql/01_construir_silver.sql && \
      duckdb db/marejada.duckdb < sql/02_comprobar_silver.sql && \
      duckdb db/marejada.duckdb < sql/03_construir_gold.sql && \
      duckdb db/marejada.duckdb < sql/04_comprobar_gold.sql)
```

RESULTADO

```
┌──────────────────────────────────────────────────┬─────────┐
│                   comprobacion                   │   ok    │
│                     varchar                      │ boolean │
├──────────────────────────────────────────────────┼─────────┤
│ Clientes: misma cantidad de fichas que en bronze │ true    │
│ Clientes: ninguna provincia por revisar          │ true    │
│ Productos: ninguna categoría por revisar         │ true    │
│ Líneas: ningún (pedido, línea) repetido          │ true    │
│ Pedidos: ninguna entrega anterior al pedido      │ true    │
│ Pedidos: 23 entregas marcadas como sospechosas   │ true    │
└──────────────────────────────────────────────────┴─────────┘

┌──────────────────────────────────────────────────────┬─────────┐
│                      comprobacion                    │   ok    │
│                        varchar                       │ boolean │
├──────────────────────────────────────────────────────┼─────────┤
│ fact_ventas: mismas líneas que silver sin cancelados │ true    │
│ fact_ventas: mismo importe que silver sin cancelados │ true    │
│ fact_pedidos: un pedido por fila, todos los de silver│ true    │
│ Hechos: ninguna clave de dimensión vacía             │ true    │
│ Hechos: todas las fechas existen en dim_fecha        │ true    │
│ dim_cliente: una sola versión válida por cliente y día │ true  │
│ Puente de marzo de 2025: 1.082.243,50 como contabilidad │ true │
└──────────────────────────────────────────────────────┴─────────┘
( duckdb db/marejada.duckdb < sql/00_cargar_bronze.sql && duckdb  <  && duckd)  0.05s user 0.01s system
4% cpu 1.349 total
```

Trece `true` y algo más de un segundo en un MacBook Air M2. Fíjate en la última línea: es el formato de `time` en **zsh**, la terminal de tu Mac, distinto del que viste en el MJD-101 del Volumen 01 (que era el de bash). zsh resume el comando medido (por eso aparece recortado) y lo que importa es el número antes de `total`: el tiempo de reloj. Los `user` y `system` tan bajos son solo los de la terminal: el trabajo lo hace cada `duckdb`, y zsh no lo suma aquí.

La barra invertida del final de cada línea dice a la terminal que el comando sigue en la línea siguiente, y `&&` encadena los comandos de forma que si uno falla, los siguientes no se ejecutan. Los paréntesis agrupan los cinco para que `time` los mida juntos.

Y al repositorio:

TÚ ESCRIBES

```
git add sql/03_construir_gold.sql sql/04_comprobar_gold.sql
git commit -m "feat: capa gold con el modelo en estrella y sus comprobaciones"
git push
```

PARTE 3

Usar el modelo

Cómo se consulta una estrella, cómo se combinan dos tablas de hechos y qué espera Power BI del modelo que acabas de construir.

11. Consultar una estrella

12. Lo que Power BI espera del modelo

11. Consultar una estrella

11.1 Siempre el mismo patrón

Todas las consultas sobre una estrella tienen la misma forma, y por eso son fáciles de escribir y de revisar:

```sql
SELECT <atributos de las dimensiones>,      -- cómo se mira
       <agregados de las medidas>           -- lo que se mide
FROM gold.fact_x AS f
JOIN gold.dim_a AS a ON a.a_key = f.a_key   -- una unión por dimensión, siempre por la clave
JOIN gold.dim_b AS b ON b.b_key = f.b_key
WHERE <filtros sobre atributos de las dimensiones>
GROUP BY <los mismos atributos del SELECT>;
```

Las uniones son siempre **de los hechos a una dimensión, por la clave subrogada**. Nunca entre dimensiones y nunca entre dos tablas de hechos (11.4). Y como cada hecho tiene una única fila en cada dimensión, ninguna unión multiplica filas: el abanico es imposible.

Ventas de 2026 por tipo de cliente y canal:

```sql
SELECT c.tipo_cliente,
       round(sum(f.importe_neto) FILTER (WHERE f.canal = 'Comercial')) AS comercial,
       round(sum(f.importe_neto) FILTER (WHERE f.canal = 'Teléfono'))  AS telefono,
       round(sum(f.importe_neto) FILTER (WHERE f.canal = 'Web B2B'))   AS web,
       round(100 * sum(f.importe_neto) FILTER (WHERE f.canal = 'Web B2B') / sum(f.importe_neto), 1) AS
pct_web
FROM gold.fact_ventas AS f
JOIN gold.dim_cliente AS c ON c.cliente_key = f.cliente_key
JOIN gold.dim_fecha   AS d ON d.fecha_key   = f.fecha_key
WHERE d.anio = 2026
GROUP BY c.tipo_cliente
ORDER BY pct_web DESC;
```

RESULTADO

tipo_cliente varchar	comercial double	telefono double	web double	pct_web double
Restaurante	1528296.0	692705.0	665093.0	23.0
Tienda de alimentación	421488.0	174054.0	175643.0	22.8
Sidrería	283855.0	107915.0	112162.0	22.3
Club / Discoteca	111521.0	49392.0	46287.0	22.3
Bar	1245397.0	529588.0	497440.0	21.9
Cafetería	321521.0	149192.0	131121.0	21.8
Hotel	1652414.0	698141.0	649262.0	21.6
Supermercado	2115446.0	890680.0	779740.0	20.6
Catering	90786.0	37623.0	26621.0	17.2

El peso de la web es casi el mismo en todos los tipos de local, entre el 20,6 % y el 23 %, salvo los caterings (17,2 %). Es una pista para Paula: la web no es cosa de un tipo de cliente concreto, y una campaña dirigida solo a los bares, por ejemplo, no tendría sentido. Una pregunta así sale en una consulta porque el tipo de cliente es un atributo de la dimensión.

11.2 Lo que ya no tienes que recordar

Compara esta consulta con la del 2.1. Ha desaparecido todo lo que había que recordar: no hay `estado <> 'Cancelado'`, no hay `cliente_maestro_id`, no hay `date_trunc` ni rangos de fechas. El año es una columna; la provincia es una columna. Esa es la promesa del modelo: **que una pregunta de negocio se escriba como una pregunta de negocio**.

11.3 Una dimensión, dos papeles

`fact_pedidos` tiene dos fechas, y las dos apuntan a `dim_fecha`. Cuando una dimensión aparece más de una vez en una tabla de hechos con significados distintos, se dice que juega varios **papeles** (*role-playing dimension*). En SQL se resuelve uniendo la dimensión dos veces, con dos alias:

SQL · EN DUCKDB

```
SELECT fp.nombre_dia AS dia_del_pedido,
       fe.nombre_dia AS dia_de_entrega,
       count(*) AS pedidos
FROM gold.fact_pedidos AS f
JOIN gold.dim_fecha AS fp ON fp.fecha_key = f.fecha_key
JOIN gold.dim_fecha AS fe ON fe.fecha_key = f.fecha_entrega_key
WHERE fp.anio = 2026
GROUP BY fp.dia_semana, fp.nombre_dia, fe.dia_semana, fe.nombre_dia
ORDER BY fp.dia_semana, pedidos DESC;
```

RESULTADO

dia_del_pedido	dia_de_entrega	pedidos
varchar	varchar	int64
lunes	martes	2621
lunes	miércoles	1738
lunes	jueves	790
martes	miércoles	2247
martes	jueves	1455
martes	viernes	731
miércoles	jueves	2091
miércoles	viernes	1473
miércoles	lunes	726
jueves	lunes	2283
jueves	viernes	2258
viernes	lunes	4263
sábado	lunes	1088
sábado	martes	238

14 rows — 3 columns

Los pedidos del viernes se entregan todos el lunes, y los del jueves se reparten entre el viernes y el lunes. No se reparte en fin de semana. Es la base del MJD-405.

11.4 Combinar dos tablas de hechos

Las ventas por línea están en `fact_ventas` y los portes en `fact_pedidos`. Para tener las dos cosas por mes hay una tentación evidente: unir las dos tablas de hechos por `pedido_id`. Es exactamente el abanico del Volumen 02 con otro nombre, y lo verás en el MJD-407. La forma correcta se llama **drill-across**: agregar cada tabla de hechos por su lado hasta el mismo nivel de una dimensión común, y unir los resultados ya agregados.

SQL · EN DUCKDB

```
WITH ventas AS (
    SELECT d.anio_mes, sum(f.importe_neto) AS ventas
    FROM gold.fact_ventas AS f
    JOIN gold.dim_fecha AS d ON d.fecha_key = f.fecha_key
    WHERE d.anio = 2025 AND d.trimestre = 1
    GROUP BY d.anio_mes
),
portes AS (
    SELECT d.anio_mes, sum(f.gastos_envio) AS portes, count(*) AS pedidos
    FROM gold.fact_pedidos AS f
    JOIN gold.dim_fecha AS d ON d.fecha_key = f.fecha_key
    WHERE d.anio = 2025 AND d.trimestre = 1 AND f.estado <> 'Cancelado'
    GROUP BY d.anio_mes
)
SELECT anio_mes, round(ventas, 2) AS ventas, portes, pedidos
FROM ventas
JOIN portes USING (anio_mes)
ORDER BY anio_mes;
```

RESULTADO

```
┌──────────┬──────────┬─────────────────┬─────────┐
│ anio_mes │  ventas  │     portes      │ pedidos │
│ varchar  │  double  │ decimal(38,2)   │  int64  │
├──────────┼──────────┼─────────────────┼─────────┤
│ 2025-01  │ 974273.96 │         4140.00 │    1697 │
│ 2025-02  │ 848369.94 │         3822.00 │    1480 │
│ 2025-03  │1063694.42 │         4920.00 │    1838 │
└──────────┴──────────┴─────────────────┴─────────┘
```

Cada tabla de hechos se agrega por su grano hasta el mes, y los dos resultados (una fila por mes cada uno) se unen por el mes. Ninguna fila se multiplica. El mes es el atributo de una **dimensión conformada** (3.6): por eso las dos tablas se pueden comparar.

IDEA CLAVE

Nunca unas dos tablas de hechos entre sí. Agrega cada una por su lado hasta los atributos de una dimensión que compartan, y une los resultados. En Power BI pasa lo mismo, y lo resuelve el propio modelo: cada tabla de hechos se relaciona con las dimensiones, nunca con la otra tabla de hechos.

12. Lo que Power BI espera del modelo

12.1 Por qué Power BI quiere una estrella

Power BI guarda los datos en un motor por columnas (VertiPaq), comprimido y en memoria: la misma idea del Volumen 01 (4.2). Ese motor está optimizado para un patrón concreto: **filtrar las dimensiones y que ese filtro viaje a los hechos** por las relaciones. Cuando eliges `marzo` en un filtro, Power BI filtra `dim_fecha`, la relación propaga el filtro a `fact_ventas` y las medidas se calculan solo sobre las filas de marzo.

Con una estrella, ese viaje es de un solo salto. Con un copo de nieve son varios. Con una tabla única gigante (todo desnormalizado en los hechos) el modelo ocupa mucho más y las medidas DAX se complican. Por eso la estrella no es una preferencia estética: es la forma para la que está construido el motor.

12.2 Relaciones

Cuando cargues gold en Power BI (Volumen 08), cada unión del 11.1 será una **relación** del modelo:

RELACIÓN	CARDINALIDAD	DIRECCIÓN DEL FILTRO
`dim_fecha[fecha_key]` → `fact_ventas[fecha_key]`	Uno a varios (1:*)	Única: de la dimensión a los hechos
`dim_cliente[cliente_key]` → `fact_ventas[cliente_key]`	Uno a varios	Única
`dim_producto[producto_key]` → `fact_ventas[producto_key]`	Uno a varios	Única
`dim_comercial[comercial_key]` → `fact_ventas[comercial_key]`	Uno a varios	Única
Y las mismas desde fecha, cliente y comercial hacia `fact_pedidos`	Uno a varios	Única

Las dimensiones en el lado "uno", los hechos en el lado "varios", y el filtro en una sola dirección. Las relaciones bidireccionales existen, pero dan resultados ambiguos y lentos: el examen PL-300 pregunta cuándo **no** usarlas.

Fíjate en una cosa que hace que esto funcione: en el lado "uno", la clave no puede repetirse. En `dim_cliente`, `cliente_key` es única aunque `cliente_id` se repita (la SCD de tipo 2). Si hubieras relacionado por `cliente_id`, Power BI se negaría a crear la relación uno a varios. La clave subrogada no era trabajo de más.

12.3 La tabla de fechas

Power BI tiene funciones de **inteligencia de tiempo** (acumulado del año, mismo periodo del año anterior...) que necesitan una tabla de fechas "marcada" como tal. Para poder marcarla, la columna de fecha tiene que cumplir:

- Un valor por día, **sin días que falten** entre el primero y el último.
- **Sin repetidos** y **sin valores en blanco**.
- Lo recomendable: **años completos**, del 1 de enero al 31 de diciembre.

`gold.dim_fecha` cumple las tres, y por eso empieza el 1 de enero de 2024, acaba el 31 de diciembre de 2026 y no tiene miembro desconocido (9.2). Las funciones de inteligencia de tiempo son en DAX lo que en el Volumen 03 escribiste con ventanas y `INTERVAL 1 YEAR`; las verás en el Volumen 08.

12.4 Los papeles en Power BI

En SQL, la dimensión fecha se une dos veces con dos alias (11.3). En Power BI, entre dos tablas solo puede haber **una relación activa**. Con `fact_pedidos` habrá dos relaciones con `dim_fecha` (por `fecha_key` y por `fecha_entrega_key`), y una de ellas quedará inactiva. Hay dos soluciones, y las dos entran en el PL-300:

SOLUCIÓN	CÓMO	CUÁNDO
Activar la relación en la medida	Una medida DAX con `USERELATIONSHIP` que usa la relación inactiva solo para ese cálculo	Cuando el segundo papel se usa en pocas medidas
Duplicar la dimensión	Una segunda tabla de fechas (por ejemplo, "Fecha de entrega") con su propia relación activa	Cuando los usuarios quieren filtrar por las dos fechas a la vez

12.5 En Fabric

En el Bloque 2 construirás esta misma capa gold dentro de Microsoft Fabric: las tablas vivirán en un *lakehouse* o un *warehouse*, el script será SQL o un notebook, y Power BI leerá las tablas directamente de OneLake con el modo **Direct Lake**, sin copiarlas. Las decisiones de este volumen (grano, hechos, dimensiones, claves, SCD) son exactamente las mismas: cambia la herramienta, no el modelo. Y el DP-600 las pregunta.

TICKETS

Gold en producción

Ocho encargos sobre el modelo: usarlo, ampliarlo y defenderlo. Guarda las consultas en sql/04-modelado/.

Tickets · Volumen 04

Soluciones · Volumen 04

Tickets · Volumen 04

Fecha de corte: **30 de septiembre de 2026**. Todas las consultas, sobre gold.

TICKET MJD-401 De: Elena Pérez · Responsable de datos

Prioridad: media

Marcos quiere un cuadro de mando de stock. Antes de construir nada, quiero tu diseño en una página: qué tabla de hechos nueva haría falta, su grano, sus dimensiones, sus medidas y cómo se pueden sumar.

ENTREGABLE

`entregables/mjd-401_diseno_stock.md`. Sin SQL: es un diseño.

TICKET MJD-402 De: Lucía Gómez · Administración y finanzas

Prioridad: alta

He oído que ya tenéis un "modelo" nuevo. Antes de fiarme: sácame con él la facturación de marzo de 2025 por provincia, con nuestro criterio, y la comparo con mi hoja.

TICKET MJD-403 De: Raquel Fernández · Jefa de ventas Cantabria

Prioridad: media

Celia lleva un año con la cartera de Héctor. ¿Vende esa cartera más o menos que cuando la llevaba él? Compárame el primer semestre de 2025 (Héctor) con el primer semestre de 2026 (Celia).

PISTA

La respuesta depende de qué clientes compares. ¿Los mismos clientes en los dos periodos, o la cartera que tenía cada uno en su momento?

TICKET MJD-404 De: Íñigo Sainz · Director comercial

Prioridad: alta

Margen bruto por categoría en 2025. Quiero saber dónde ganamos dinero de verdad, no solo dónde vendemos más.

TICKET MJD-405 De: Marcos Diego · Logística y almacén

Prioridad: media

Los clientes se quejan de que los pedidos del viernes tardan mucho. ¿Es verdad? Plazo medio por día de la semana en que se hace el pedido, en 2026, y si cambia entre almacenes.

TICKET MJD-406 De: Íñigo Sainz · Director comercial

Prioridad: media

Quiero analizar las ventas por gama de precio: básicos (menos de 15 €), gama media (de 15 a 50 €) y premium (más de 50 €), por precio de tarifa. Y lo quiero poder usar en Power BI, no pedírtelo cada vez.

PISTA

"Lo quiero usar siempre" es una pista sobre **dónde** va esta regla.

TICKET MJD-407 De: Elena Pérez · Responsable de datos

Prioridad: alta

Un compañero de otro equipo ha hecho esta consulta para un informe de marzo de 2025 y los portes le salen el doble de lo que dice contabilidad. ¿Qué le pasa y cómo se arregla? Explícaselo de forma que no vuelva a pasar.

SQL

```
SELECT round(sum(v.importe_neto), 2) AS ventas,
       sum(p.gastos_envio)           AS portes
FROM gold.fact_ventas AS v
JOIN gold.fact_pedidos AS p USING (pedido_id)
WHERE v.fecha_key BETWEEN 20250301 AND 20250331;
```

TICKET MJD-408 De: Carmen Ruiz · Directora general

Prioridad: media

En un informe he visto una fila que pone "Cliente desconocido". ¿Qué es eso, cuánto dinero es y qué vamos a hacer?

Soluciones · Volumen 04

MJD-401 · Diseño de una tabla de hechos de stock

No hace falta SQL, pero sí todo lo de la Parte 1. El documento, en el formato que Elena pide:

FICHERO · entregables/mjd-401_diseno_stock.md

```
# Diseño · fact_stock

Proceso: el inventario de los almacenes.
Tipo: foto periódica (periodic snapshot), una foto por día.

## Grano
Una fila es: las unidades de un producto en un almacén al cierre de un día.

## Dimensiones
- dim_fecha (el día de la foto)
- dim_producto
- almacén: dimensión degenerada (Guarnizo, Llanera)

## Medidas
- unidades: SEMIADITIVA. Se suma entre productos y almacenes,
  nunca entre días. Para un mes: la del último día (o la media).
- valor_coste = unidades × coste: igual que unidades.

## Decisiones pendientes
- Origen: hoy solo existe el inventario del 30/09/2026. Para tener
  una foto diaria hace falta que el programa de almacén la exporte
  cada noche.
- SKU sin producto en el ERP (9 OBS-xxxx): producto_key = -1.
```

REVISIÓN DE ELENA

Muy bien, y lo que más me gusta es la línea de "semiaditiva". Es justo lo que un cuadro de mando de stock hace mal la primera vez: alguien pone una tarjeta con "unidades" y Power BI, por defecto, **suma** las de todos los días del mes. Treinta fotos de 21.000 unidades darían 630.000. En el Volumen 08 escribirás la medida DAX que coge solo el último día; tener la decisión escrita ahora es lo que evitará el error.

Y la de las decisiones pendientes es la más profesional del documento: has visto que el diseño necesita algo que hoy no existe (una exportación diaria). Un diseño que esconde lo que falta no se puede construir.

MJD-402 · Marzo de 2025 desde el modelo

El criterio de contabilidad sale entero de `fact_pedidos`: mes de entrega, estados facturables, importe más portes. Nada de quitar duplicados, ni de agregar líneas a mano para evitar el abanico, ni de normalizar provincias. Todo eso lo hizo el modelo.

SQL · EN DUCKDB

```
SELECT c.provincia,
       round(sum(f.importe_neto + f.gastos_envio), 2) AS base_imponible
FROM gold.fact_pedidos AS f
JOIN gold.dim_cliente AS c ON c.cliente_key = f.cliente_key
JOIN gold.dim_fecha   AS d ON d.fecha_key   = f.fecha_entrega_key
WHERE d.anio_mes = '2025-03'
  AND f.estado IN ('Entregado', 'Devuelto parcial')
GROUP BY c.provincia
ORDER BY c.provincia;
```

RESULTADO

```
┌─────────────┬─────────────────┐
│  provincia  │ base_imponible  │
│   varchar   │     double      │
├─────────────┼─────────────────┤
│ Asturias    │       205455.25 │
│ Bizkaia     │         9992.87 │
│ Burgos      │       116563.83 │
│ Cantabria   │       678588.45 │
│ Desconocida │         6305.36 │
│ Palencia    │        65337.74 │
└─────────────┴─────────────────┘
```

Y la hoja de Lucía, para comparar:

SQL · EN DUCKDB

```
SELECT "Provincia" AS provincia, "Base imponible (€)" AS contabilidad
FROM read_xlsx('datos/contabilidad/facturacion_mensual_2024-2026.xlsx', range = 'A4:E1000')
WHERE "Año" = 2025 AND "Mes" = 'marzo'
ORDER BY provincia;
```

RESULTADO

```
┌─────────────┬──────────────┐
│ provincia   │ contabilidad │
│  varchar    │    double    │
├─────────────┼──────────────┤
│ Asturias    │    205455.25 │
│ Bizkaia     │      9992.87 │
│ Burgos      │    116563.83 │
│ Cantabria   │    684893.81 │
│ Palencia    │     65337.74 │
└─────────────┴──────────────┘
```

Cuatro provincias iguales al céntimo. Cantabria y la fila `Desconocida` son la misma historia del MJD-204: los 6.305,36 € de los clientes borrados, que contabilidad asigna a Cantabria y el modelo deja en el miembro desconocido. Sumados, coinciden. Fíjate en la unión con la fecha: `fecha_entrega_key`, el segundo papel de `dim_fecha` (11.3).

REVISIÓN DE ELENA

Esto es lo que da confianza en un modelo: reproducir con él la cifra más discutida de la empresa, y que cuadre. Díselo a Lucía tal cual, y cuéntale que la comprobación ya está automatizada (la última de `04_comprobar_gold.sql`): cada vez que se reconstruya gold, alguien comprobará marzo de 2025 por ella.

Y la fila "Desconocida" no se esconde: se explica. El MJD-408 va de eso.

MJD-403 · La cartera de Celia frente a la de Héctor

La pista apuntaba a una ambigüedad real. Hay dos preguntas distintas:

1. **Los mismos clientes**: los clientes que hoy lleva Celia, ¿compran más o menos ahora que cuando los llevaba Héctor? Es una comparación justa del comportamiento de los clientes.
2. **La cartera de cada uno en su momento**: lo que vendía la cartera asignada a Héctor en el primer semestre de 2025 frente a lo que vende la asignada a Celia en el de 2026. Incluye a los clientes que se fueron y a los nuevos.

Con la SCD de tipo 2, la segunda pregunta sale directamente de `comercial_asignado` (la versión vigente en cada venta). La primera se responde filtrando por los clientes cuya versión **actual** es de Celia:

SQL · EN DUCKDB

```
WITH clientes_de_celia AS (
    SELECT cliente_id
    FROM gold.dim_cliente
    WHERE es_version_actual AND comercial_asignado_id = 13
)
SELECT d.anio,
       c.comercial_asignado,
       count(DISTINCT c.cliente_id) AS clientes_con_compras,
       round(sum(f.importe_neto)) AS ventas_semestre
FROM gold.fact_ventas AS f
JOIN gold.dim_cliente AS c ON c.cliente_key = f.cliente_key
JOIN gold.dim_fecha   AS d ON d.fecha_key   = f.fecha_key
WHERE c.cliente_id IN (SELECT cliente_id FROM clientes_de_celia)
  AND d.mes <= 6 AND d.anio IN (2025, 2026)
GROUP BY d.anio, c.comercial_asignado
ORDER BY d.anio;
```

RESULTADO

anio	comercial_asignado	clientes_con_compras	ventas_semestre
int64	varchar	int64	double
2025	Héctor Bustamante Lavín	86	596416.0
2026	Celia Gómez Sáiz	88	622559.0

SQL · EN DUCKDB

```
SELECT d.anio,
       c.comercial_asignado,
       count(DISTINCT c.cliente_id) AS clientes_con_compras,
       round(sum(f.importe_neto)) AS ventas_semestre
FROM gold.fact_ventas AS f
JOIN gold.dim_cliente AS c ON c.cliente_key = f.cliente_key
JOIN gold.dim_fecha   AS d ON d.fecha_key   = f.fecha_key
WHERE c.comercial_asignado_id IN (7, 13)
  AND d.mes <= 6 AND d.anio IN (2025, 2026)
GROUP BY d.anio, c.comercial_asignado
ORDER BY d.anio;
```

RESULTADO

anio	comercial_asignado	clientes_con_compras	ventas_semestre
int64	varchar	int64	double
2025	Héctor Bustamante Lavín	86	596416.0
2026	Celia Gómez Sáiz	88	622559.0

En este caso las dos preguntas dan exactamente lo mismo, y eso también es información: toda la cartera de Héctor (90 clientes con historial) pasó a Celia, y los otros 10 clientes que hoy lleva ella se dieron de alta después o no habían comprado aún; ninguno compró con Héctor ni en el primer semestre de 2025. Las cifras: la cartera vendió 596.416 € en el primer semestre de 2025 con Héctor y 622.559 € en el de 2026 con Celia, un 4,4 % más.

REVISIÓN DE ELENA

Bien planteado: has visto las dos preguntas, has calculado las dos y has comprobado que coinciden. Eso es lo que hay que contarle a Raquel, porque si no coincidieran, cada una diría una cosa distinta y alguien elegiría la que le conviene.

Lo que le añadiría, para ser justo con los dos: una cifra sin su referencia no evalúa a nadie. Las ventas de Cantabria crecen un 13,4 % entre esos dos semestres (4,21 millones a 4,77, con la misma consulta y `provincia = 'Cantabria'` en lugar del filtro de comercial). Frente a eso, un 4,4 % es poco. Pero antes de que nadie saque conclusiones sobre Celia: la cartera de Héctor está en Cantabria Occidental, con su propia mezcla de clientes, y lo justo es compararla con las demás carteras de esa zona, no con toda la provincia. Dale a Raquel la cifra, la referencia y la advertencia.

MJD-404 · Margen bruto por categoría

El margen es una medida no aditiva (3.3): se calcula a partir de dos aditivas, al final.

SQL · EN DUCKDB

```
SELECT p.categoria,
       round(sum(f.importe_neto)) AS ventas,
       round(sum(f.coste_estimado)) AS coste_estimado,
       round(sum(f.importe_neto) - sum(f.coste_estimado)) AS margen,
       round(100 * (1 - sum(f.coste_estimado) / sum(f.importe_neto)), 1) AS margen_pct
FROM gold.fact_ventas AS f
JOIN gold.dim_producto AS p ON p.producto_key = f.producto_key
JOIN gold.dim_fecha    AS d ON d.fecha_key    = f.fecha_key
WHERE d.anio = 2025
  AND f.producto_key <> -1
GROUP BY p.categoria
ORDER BY margen DESC;
```

RESULTADO

categoria varchar	ventas double	coste_estimado decimal(38,0)	margen double	margen_pct double
Vinos	3947915.0	2576130	1371785.0	34.7
Cervezas	3161161.0	2356965	804196.0	25.4
Alimentación	2370009.0	1765282	604726.0	25.5
Refrescos	2099874.0	1587938	511936.0	24.4
Desechables	986518.0	634986	351532.0	35.6
Licores y destilados	948664.0	614254	334410.0	35.3
Café e infusiones	832078.0	519921	312157.0	37.5
Limpieza e higiene	658790.0	444504	214286.0	32.5
Aguas	614210.0	429073	185137.0	30.1

Las dos categorías que más venden, vinos y cervezas, son también las que más margen dejan en euros, pero con porcentajes muy distintos: un 34,7 % el vino y un 25,4 % la cerveza. Café, desechables y licores tienen los márgenes más altos en porcentaje con ventas mucho menores.

El `producto_key <> -1` excluye las líneas de productos desconocidos, que no tienen coste: dentro, pondrían ventas sin coste y el margen saldría algo inflado.

REVISIÓN DE ELENA

La cifra está bien calculada; lo que no te dejo enviar es sin la advertencia del 9.3, y en la primera línea del correo: "Margen estimado con el coste de compra actual (2026). Para 2025 está subestimado, porque los precios de venta de 2025 eran más bajos y el coste que se aplica es el de hoy". Íñigo va a tomar decisiones de negocio con este número.

La pregunta que te hará después: ¿y el margen real? Para eso Compras tendría que darnos el historial de costes, y el coste sería un atributo de tipo 2 en `dim_producto` (o, mejor, el coste de cada línea en la tabla de hechos, calculado en el momento de la venta). Propónselo.

MJD-405 · ¿Tardan más los pedidos del viernes?

El plazo medio por almacén y día de la semana del pedido, con `dim_fecha` en el papel de fecha del pedido:

SQL · EN DUCKDB

```
SELECT f.almacen,
       d.nombre_dia AS dia_del_pedido,
       count(*) AS pedidos,
       round(avg(f.dias_entrega), 2) AS dias_medios
FROM gold.fact_pedidos AS f
JOIN gold.dim_fecha AS d ON d.fecha_key = f.fecha_key
WHERE d.anio = 2026
  AND f.fecha_entrega_key IS NOT NULL
GROUP BY f.almacen, d.dia_semana, d.nombre_dia
ORDER BY f.almacen, d.dia_semana;
```

RESULTADO

almacen varchar	dia_del_pedido varchar	pedidos int64	dias_medios double
Guarnizo	lunes	4337	1.64
Guarnizo	martes	3716	1.66
Guarnizo	miércoles	3613	2.02
Guarnizo	jueves	3777	2.5
Guarnizo	viernes	3557	3.0
Guarnizo	sábado	1111	2.18
Llanera	lunes	812	1.66
Llanera	martes	717	1.66
Llanera	miércoles	677	2.01
Llanera	jueves	764	2.53
Llanera	viernes	706	3.0
Llanera	sábado	215	2.15

12 rows 4 columns

Sí: los pedidos del viernes tardan 3 días de media frente a 1,6 los del lunes, en los dos almacenes por igual. Pero el 11.3 ya contaba por qué: no se reparte en fin de semana, así que todo lo pedido el viernes llega el lunes. En días laborables, el viernes tarda lo mismo que cualquier otro día.

REVISIÓN DE ELENA

Correcto, y bien leído: el dato confirma la queja y a la vez la explica. Para Marcos hay dos respuestas posibles y no te toca a ti elegir: o se acepta (y se comunica a los clientes que lo pedido el viernes llega el lunes), o se estudia repartir los sábados. Dale el número que necesita para decidir: cuántos pedidos entran los viernes.

Una mejora del modelo que sale de aquí: si `dim_fecha` tuviera una columna con el número de día laborable, el plazo en días laborables sería una resta. Apúntalo para cuando se añadan los festivos (6.2).

MJD-406 · Gamas de precio

La pista hablaba de **dónde** va la regla. Hay dos sitios posibles:

- En cada consulta, con un `CASE` sobre el precio. Funciona para hoy, pero cada informe tendría su propio `CASE`, y el día que Íñigo cambie el límite de 15 € a 20 €, habrá que encontrarlos todos.
- En la dimensión, como un atributo más del producto. Se escribe una vez, se ve en Power BI como cualquier otra columna (filtros, ejes, leyendas) y se cambia en un solo sitio.

La segunda, claramente: los atributos descriptivos van en la dimensión. Es un cambio en `sql/03_construir_gold.sql`, en la consulta de `dim_producto`. Aquí lo pruebas como consulta, sin tocar la tabla:

SQL · EN DUCKDB

```
WITH producto_con_gama AS (
    SELECT *,
           CASE WHEN precio_lista IS NULL THEN 'Desconocida'
                WHEN precio_lista < 15     THEN '1. Básico'
                WHEN precio_lista <= 50    THEN '2. Gama media'
                ELSE                            '3. Premium'
           END AS gama_precio
    FROM gold.dim_producto
)
SELECT p.gama_precio,
       count(DISTINCT p.producto_key) AS productos,
       round(sum(f.importe_neto)) AS ventas_2025,
       round(100 * sum(f.importe_neto) / sum(sum(f.importe_neto)) OVER (), 1) AS pct
FROM gold.fact_ventas AS f
JOIN producto_con_gama AS p ON p.producto_key = f.producto_key
JOIN gold.dim_fecha AS d ON d.fecha_key = f.fecha_key
WHERE d.anio = 2025
GROUP BY p.gama_precio
ORDER BY p.gama_precio;
```

RESULTADO

gama_precio	productos	ventas_2025	pct
varchar	int64	double	double
1. Básico	66	2569360.0	16.4
2. Gama media	112	8608251.0	55.1
3. Premium	49	4441607.0	28.4
Desconocida	1	158.0	0.0

Para dejarlo permanente, se añade la columna en el `SELECT` de `dim_producto` del script (con un `NULL` más en el miembro desconocido… o mejor `'Desconocida'`, como las demás columnas de texto) y se reconstruye gold. Las comprobaciones dirán si algo se ha roto.

REVISIÓN DE ELENA

Bien razonado: la pregunta no era SQL, era de diseño. Tres detalles que miraría al revisar el cambio en el script:

- **El límite exacto.** "De 15 a 50" deja dudas en el 15 y en el 50. Has puesto el 15 en gama media y el 50 también. Pregúntaselo a Íñigo y escríbelo en un comentario del script.
- **El precio de tarifa cambia.** La gama de un producto puede cambiar si sube su precio. ¿Tipo 1 o tipo 2? Para una clasificación comercial, casi siempre tipo 1 (la gama de hoy), pero es una decisión, y es de Íñigo.
- **Los números del nombre** (`1. Básico`) son un truco para que salgan ordenados. En Power BI hay una forma más limpia (ordenar una columna por otra), que verás en el Volumen 08.

MJD-407 · Los portes salen el doble

La consulta une dos tablas de hechos por `pedido_id`. `fact_ventas` tiene una fila por línea y `fact_pedidos` una por pedido, así que cada pedido se repite tantas veces como líneas tiene, y sus portes con él. Es el abanico del Volumen 02 (8.6), ahora entre dos tablas de hechos de grano distinto:

SQL · EN DUCKDB

```
SELECT round(sum(v.importe_neto), 2) AS ventas,
       sum(p.gastos_envio)           AS portes
FROM gold.fact_ventas AS v
JOIN gold.fact_pedidos AS p USING (pedido_id)
WHERE v.fecha_key BETWEEN 20250301 AND 20250331;
```

RESULTADO

```
┌─────────────┬──────────────────┐
│   ventas    │     portes       │
│   double    │ decimal(38,2)    │
├─────────────┼──────────────────┤
│ 1063694.42  │       10752.00   │
└─────────────┴──────────────────┘
```

Las ventas están bien (vienen del grano de las líneas), pero los portes salen 10.752 € en lugar de 4.920. Lo correcto es el *drill-across* del 11.4: cada cosa agregada en su tabla de hechos, y unidas después:

SQL · EN DUCKDB

```
SELECT (SELECT round(sum(importe_neto), 2) FROM gold.fact_ventas
        WHERE fecha_key BETWEEN 20250301 AND 20250331) AS ventas,
       (SELECT sum(gastos_envio) FROM gold.fact_pedidos
        WHERE fecha_key BETWEEN 20250301 AND 20250331
          AND estado <> 'Cancelado') AS portes;
```

RESULTADO

```
┌─────────────┬──────────────────┐
│   ventas    │     portes       │
│   double    │ decimal(38,2)    │
├─────────────┼──────────────────┤
│ 1063694.42  │        4920.00   │
└─────────────┴──────────────────┘
```

Fíjate en el `estado <> 'Cancelado'` de los portes. `fact_ventas` ya no tiene cancelados; `fact_pedidos` sí. Para que las dos cifras hablen de lo mismo, el filtro tiene que ser el mismo en las dos.

REVISIÓN DE ELENA

Bien explicado. Para que tu compañero no vuelva a caer, dale la regla y no solo la consulta: **dos tablas de hechos no se unen entre sí; se agregan cada una por su lado y se juntan por una dimensión común**. Y la forma de detectarlo él solo: si al añadir una tabla a una consulta una cifra que ya tenías cambia, la unión está multiplicando filas.

Y un detalle que acabas de descubrir y merece una línea en la documentación de gold: `fact_ventas` excluye cancelados y `fact_pedidos` no. Es una decisión correcta (4.4), pero quien combine las dos tiene que saberla.

MJD-408 · ¿Quién es el cliente desconocido?

SQL · EN DUCKDB

```
SELECT d.anio,
       count(DISTINCT f.pedido_id) AS pedidos,
       round(sum(f.importe_neto), 2) AS ventas
FROM gold.fact_ventas AS f
JOIN gold.dim_fecha AS d ON d.fecha_key = f.fecha_key
WHERE f.cliente_key = -1
GROUP BY d.anio
ORDER BY d.anio;
```

RESULTADO

```
┌────────┬─────────┬──────────┐
│ anio   │ pedidos │  ventas  │
│ int64  │  int64  │  double  │
├────────┼─────────┼──────────┤
│  2024  │     143 │ 79724.31 │
│  2025  │      76 │ 46053.52 │
└────────┴─────────┴──────────┘
```

SQL · EN DUCKDB

```
SELECT round(100 * sum(importe_neto) FILTER (WHERE cliente_key = -1) / sum(importe_neto), 2) AS
pct_ventas_sin_cliente
FROM gold.fact_ventas;
```

RESULTADO

```
┌──────────────────────┐
│ pct_ventas_sin_cliente │
│         double         │
├──────────────────────┤
│                  0.3 │
└──────────────────────┘
```

La respuesta a Carmen: "Son ventas reales a seis clientes de Cantabria que dejaron de comprar y que alguien borró del sistema de clientes. Suman unos 126.000 € desde 2024, el 0,3 % de las ventas, y en 2026 ya no hay ninguna. No se ha perdido dinero: se ha perdido la ficha. Lucía tiene sus datos, y vamos a recuperarlos."

El "vamos a recuperarlos" es un plan concreto: pedir a Lucía los datos de los seis clientes, añadirlos a silver como una tabla de correspondencias (como las de provincias), y reconstruir. Los pedidos dejarán de apuntar a `-1` y la fila "Cliente desconocido" desaparecerá de los informes.

REVISIÓN DE ELENA

Perfecto para una directora general: qué es, cuánto es, si es grave y qué se va a hacer, en cuatro frases. El "no se ha perdido dinero, se ha perdido la ficha" es exactamente el tipo de frase que tranquiliza sin esconder nada.

Y fíjate en por qué has podido contestar en cinco minutos: porque el modelo **no escondió** esos pedidos. Con un `INNER JOIN` en la construcción de gold, habrían desaparecido sin dejar rastro y nadie habría preguntado nunca. El miembro desconocido es una decisión de diseño que existe para que estas conversaciones ocurran.

APÉNDICES

Para tener a mano

La chuleta del modelado dimensional y lo que viene en el siguiente volumen.

A. Chuleta de modelado

B. Qué viene en el Volumen 05

A. Chuleta de modelado

Las decisiones, en orden

PASO	PREGUNTA	EN MAREJADA
1	¿Qué proceso de negocio?	Ventas; pedidos
2	¿Cuál es el grano? (una frase)	Una línea de pedido no cancelado; un pedido
3	¿Qué dimensiones tienen un valor único para ese grano?	Fecha, cliente, producto, comercial
4	¿Qué medidas, y de qué tipo?	Importe y cantidad (aditivas); portes (aditiva, otro grano)
5	¿Qué atributos cambian y qué historia importa?	Comercial asignado: SCD tipo 2
6	¿Qué pasa con lo que no encaja?	Miembro desconocido `-1`

Vocabulario

TÉRMINO	QUÉ ES
Tabla de hechos	Lo que se mide de un proceso; muchas filas, claves y números
Dimensión	El contexto: quién, qué, cuándo, dónde; pocas filas, muchos textos
Grano	Lo que representa una fila de la tabla de hechos, en una frase
Medida aditiva / semiaditiva / no aditiva	Se suma por todo / por todo menos el tiempo / por nada
Estrella / copo de nieve	Dimensiones en una tabla / normalizadas en varias
Clave subrogada	Clave propia del modelo, sin significado fuera de él
Miembro desconocido	La fila `-1` de cada dimensión, para los hechos sin pareja
Dimensión degenerada	Un atributo que se queda en la tabla de hechos (`pedido_id`)
Dimensión conformada	La misma dimensión compartida por varias tablas de hechos
Role-playing	Una dimensión que juega varios papeles (fecha de pedido y de entrega)
SCD tipo 1 / tipo 2	Sobrescribir / añadir una fila con fechas de validez
Drill-across	Agregar dos tablas de hechos por separado y unir por una dimensión común
Matriz de bus	Procesos × dimensiones: el mapa del modelo

Patrones de SQL

NECESITAS	ESCRIBE
Clave subrogada	`row_number() OVER (ORDER BY clave_natural)`

NECESITAS	ESCRIBE
Miembro desconocido	`... UNION ALL SELECT -1, NULL, 'Desconocido', ...`
Clave de fecha	`CAST(strftime(fecha, '%Y%m%d') AS INTEGER)`
Búsqueda de clave	`LEFT JOIN dim ON dim.natural = s.natural` y `coalesce(dim.key, -1)`
Búsqueda en SCD tipo 2	`... AND s.fecha BETWEEN dim.valido_desde AND dim.valido_hasta`
Versiones que se solapan	`a.desde <= b.hasta AND b.desde <= a.hasta`
Consulta a una estrella	Hechos + una unión por dimensión, por la clave + `GROUP BY` atributos
Drill-across	Una CTE por tabla de hechos, agregada al mismo nivel, y `JOIN` por el atributo común

B. Qué viene en el Volumen 05

Con el Volumen 04 se cierra el SQL del Bloque 0. El Volumen 05 cambia de lenguaje: **Python**, que usarás para todo lo que SQL no hace bien (leer APIs, automatizar, y todo el machine learning del máster).

- El lenguaje desde cero, pero al ritmo de alguien que ya programa en SQL y vio JavaScript: tipos, listas, diccionarios, funciones, control de flujo, errores.
- Los notebooks de Jupyter en VS Code, con el entorno de uv del Volumen 01.
- Ficheros, rutas y módulos: cómo se organiza un proyecto de Python.
- Python y DuckDB: lanzar consultas a gold desde Python y recorrer los resultados.
- Tickets de Marejada resueltos en Python, para que veas qué cambia y qué no.

ANTES DE PASAR AL VOLUMEN 05

Tienes que poder hacer esto sin mirar: declarar el grano de una tabla de hechos en una frase, explicar por qué los portes no van en `fact_ventas`, decir qué tipo de SCD usarías para el comercial asignado y para una errata en el nombre de un cliente, y escribir una consulta sobre la estrella sin consultar el volumen. Y `sql/03_construir_gold.sql` y `sql/04_comprobar_gold.sql` en tu repositorio, con sus siete `true`.